RV32IMAC System-on-Chip Datasheet

KV32 Development Team

Version 1.0, 2026-03-14

Table of Contents

1. Introduction . . . 2
   1.1. Overview . . . 2
   1.2. Key Features . . . 2
   1.3. Document Organization . . . 3
2. System Architecture . . . 4
   2.1. Block Diagram . . . 4
   2.2. System Components . . . 6
      2.2.1. Processor Core (kv32_core) . . . 6
      2.2.2. Instruction Cache (kv32_icache) . . . 6
      2.2.3. Data Cache (kv32_dcache) . . . 7
      2.2.4. Physical Memory Attributes (PMA) . . . 7
      2.2.5. Memory Subsystem . . . 7
      2.2.6. Peripheral Subsystem . . . 8
      2.2.7. Interrupt System . . . 8
      2.2.8. Power Manager (kv32_pm) . . . 8
3. Memory Map . . . 10
   3.1. Address Space Overview . . . 10
   3.2. Address Decode Rules . . . 10
4. Processor Core Specification . . . 12
   4.1. RV32IMAC ISA Support . . . 12
   4.2. Pipeline Architecture . . . 12
      4.2.1. Instruction Buffer (IB) . . . 13
      4.2.2. Store Buffer (SB) . . . 13
      4.2.3. Branch Prediction and Return Address Stack . . . 13
      4.2.4. Forwarding and Hazards . . . 14
   4.3. Performance Characteristics . . . 14
   4.4. CSR (Control and Status Registers) . . . 15
   4.5. Exception and Interrupt Handling . . . 16
      4.5.1. Exception Types . . . 16
      4.5.2. Interrupt Types . . . 17
      4.5.3. Trap Handling . . . 17
5. Instruction Cache Specification . . . 19
   5.1. Overview . . . 19
   5.2. Cache Organization . . . 19
   5.3. Cache Operations . . . 19

5.3.1. Normal Fetch (Cache Hit) . . . 19
5.3.2. Cache Miss . . . 20
5.3.3. Bypass Mode . . . 20
5.4. Cache Management Operations (CMO) . . . 20
5.5. Performance Counters (Simulation Only) . . . 21
6. Data Cache Specification . . . 22
6.1. Overview . . . 22
6.2. Cache Organisation . . . 22
6.3. Cache Operations . . . 23
6.3.1. Load Hit . . . 23
6.3.2. Load Miss — Clean Victim . . . 23
6.3.3. Load Miss — Dirty Victim (Write-Back Mode) . . . 23
6.3.4. Store Hit . . . 23
6.3.5. Store Miss . . . 24
6.3.6. PMA Bypass (Non-Cacheable Access) . . . 24
6.4. Cache Management Operations (CMO) . . . 24
6.5. Performance Counters (Simulation Only) . . . 24
7. Peripheral Specifications . . . 26
7.1. UART (Universal Asynchronous Receiver/Transmitter) . . . 26
7.1.1. Overview . . . 26
7.1.2. Register Map . . . 26
7.1.3. Baud Rate Configuration . . . 27
7.1.4. Interrupt Behavior . . . 28
7.1.5. External Pins . . . 28
7.2. SPI (Serial Peripheral Interface) . . . 28
7.2.1. Overview . . . 28
7.2.2. Register Map . . . 29
7.2.3. SPI Modes . . . 30
7.2.4. Clock Divider Examples (100 MHz System Clock) . . . 30
7.2.5. Usage Flow . . . 30
7.2.6. External Pins . . . 31
7.3. I2C (Inter-Integrated Circuit) . . . 31
7.3.1. Overview . . . 31
7.3.2. Register Map . . . 32
7.3.3. Clock Divider Examples (100 MHz System Clock) . . . 32
7.3.4. Usage Flow . . . 33
7.3.5. External Pins . . . 33

7.4. DMA (Direct Memory Access Controller) . . . 34
7.4.1. Overview . . . 34
7.4.2. Register Map . . . 34
7.4.3. Transfer Modes . . . 36
7.4.4. Interrupt Behavior . . . 37
7.4.5. Performance Monitoring . . . 37
7.4.6. AXI Interface . . . 37
7.5. GPIO (General-Purpose I/O) . . . 38
7.5.1. Overview . . . 38
7.5.2. Register Map . . . 38
7.5.3. Interrupt Behavior . . . 39
7.5.4. External Pins . . . 39
7.6. Timer/PWM Controller . . . 40
7.6.1. Overview . . . 40
7.6.2. Register Map . . . 40
7.6.3. Timer Modes . . . 41
7.6.4. Timing Example (100 MHz clock, 1 kHz interrupt) . . . 42
7.6.5. Interrupt Behavior . . . 42
7.6.6. External Pins . . . 42
7.7. Watchdog Timer (WDT) . . . 42
7.7.1. Overview . . . 43
7.7.2. Register Map . . . 43
7.7.3. Operation . . . 43
7.7.4. Interrupt Behavior . . . 44
7.7.5. External Pins . . . 44
7.8. CLINT (Core-Local Interruptor) . . . 44
7.8.1. Overview . . . 44
7.8.2. Register Map . . . 45
7.8.3. Timer Operation . . . 45
7.8.4. Software Interrupt Operation . . . 45
7.9. PLIC (Platform-Level Interrupt Controller) . . . 46
7.9.1. Overview . . . 46
7.9.2. Register Map . . . 46
7.9.3. Interrupt Sources . . . 47
7.9.4. Interrupt Flow . . . 47
7.9.5. Priority Arbitration . . . 48
7.10. Magic Device (Simulation Only) . . . 48

7.10.1. Overview . . . 48
7.10.2. Register Map . . . 49
7.10.3. AXI Response Behavior . . . 49
7.10.4. Exit Code Encoding . . . 50
8. AXI Interconnect and Bus Protocol . . . 51
8.1. AXI4-Lite Overview . . . 51
8.2. Crossbar Architecture . . . 51
8.2.1. Address Decode Logic . . . 51
8.2.2. Error Responses . . . 52
8.3. AXI4 Memory Interface . . . 52
8.3.1. I-Cache Burst Transactions . . . 52
8.3.2. D-Cache Burst Transactions . . . 52
8.3.3. Data Access Transactions . . . 54
8.4. Bus Performance . . . 54
9. Electrical and Timing Characteristics . . . 55
9.1. Clock and Reset . . . 55
9.2. Timing Parameters . . . 55
9.3. Power Consumption (Estimated) . . . 55
9.4. Environmental Specifications . . . 56
10. Programming Guide . . . 57
10.1. Boot Sequence . . . 57
10.2. Memory Initialization . . . 57
10.3. Interrupt Handling . . . 57
10.3.1. Setting Up Interrupts . . . 57
10.3.2. Trap Handler Example . . . 58
10.4. Peripheral Usage Examples . . . 59
10.4.1. UART Echo Program . . . 59
10.4.2. SPI Flash Read . . . 59
10.4.3. I2C EEPROM Write . . . 59
10.4.4. DMA Memory Copy . . . 60
10.5. Performance Optimization Tips . . . 60
10.6. Debugging . . . 61
10.6.1. Simulation Debug . . . 61
10.6.2. Waveform Analysis . . . 61
10.6.3. Instruction Trace Comparison . . . 61
11. Conclusion . . . 63
Appendix A: Register Quick Reference . . . 64

A.1. CLINT Registers . . . 64
A.2. PLIC Registers . . . 64
A.3. UART Registers . . . 64
A.4. SPI Registers . . . 65
A.5. I2C Registers . . . 65
A.6. DMA Global Registers . . . 66
A.7. GPIO Registers (Bank 0 shown; bank *b* at `BASE + group + bx4`) . . . 66
A.8. Timer Registers (Channel 0 shown; channel *n* at `BASE + nx0x20`) . . . 67
Appendix B: Interrupt Vector Table . . . 68
Appendix C: Pin Descriptions . . . 69
C.1. Clock and Reset . . . 69
C.2. UART Interface . . . 69
C.3. SPI Interface . . . 69
C.4. I2C Interface . . . 69
C.5. GPIO Interface . . . 70
C.6. Timer / PWM Interface . . . 70
Appendix D: RISC-V Architectural Extensions and KV32-Specific Features . . . 72
D.1. Scope . . . 72
D.2. ISA and Privileged Architecture Summary . . . 72
D.3. Standard Architectural Features Implemented . . . 73
D.3.1. Machine-Mode Trap Architecture . . . 73
D.3.2. Standard Counters . . . 74
D.3.3. Atomic Extension Scope . . . 74
D.3.4. Compressed Instruction Support . . . 74
D.4. Standard Optional Extensions with KV32-Specific Behavior . . . 74
D.4.1. `FENCE.I` (`Zifencei`) . . . 74
D.4.2. Cache Block Management (`Zicbom`-style) . . . 74
D.5. KV32-Specific Non-Standard Architectural Features . . . 75
D.5.1. Runtime-Configurable PMA CSRs . . . 75
D.5.2. Stack-Guard CSRs and Stack-Overflow Trap . . . 75
D.5.3. Cacheability Fallback Rule . . . 76
D.5.4. Machine-Mode-Only System Profile . . . 76
D.6. Features That Are Platform-Specific, Not ISA Extensions . . . 76
D.7. Portability Guidance . . . 77
Appendix E: Revision History . . . 78

This document provides comprehensive technical specifications for the RV32IMAC System-on-Chip (SoC), a high-performance 32-bit RISC-V processor platform. The SoC features a 5-stage pipelined RV32IMAC core with separate instruction and data caches, AXI4/AXI4-Lite interconnect, and a rich set of peripherals including UART, SPI, I2C, DMA, GPIO, timer/PWM, and interrupt controller. Runtime-configurable Physical Memory Attributes (PMA) control per-region cacheability for both caches. Designed for embedded applications, the SoC supports both RTL simulation and FPGA implementation.

Chapter 1. Introduction

1.1. Overview

The RV32IMAC SoC is a complete embedded processor system implementing the RISC-V RV32IMAC instruction set architecture with Machine mode (M-mode) privilege support. The system integrates:

- **RV32IMAC Processor Core**: 5-stage pipeline with RVC expander, instruction cache, data cache, store buffer, and instruction buffer
- **Memory Subsystem**: 2MB external RAM with AXI4 interface, configurable instruction and data caches
- **Physical Memory Attributes (PMA)**: 8 runtime-configurable regions controlling I-cache (X bit) and D-cache (C bit) cacheability per address range
- **Interconnect**: AXI4-Lite crossbar interconnect supporting multiple master and slave devices
- **Standard Peripherals**: UART, SPI, I2C, DMA with FIFO-buffered interfaces
- **Interrupt System**: CLINT (Core-Local Interrupt) for timer/software interrupts, PLIC (Platform-Level Interrupt Controller) for external peripherals
- **Debug Support**: Magic device for simulation control and console I/O

1.2. Key Features

Feature	Specification
ISA	RV32IMAC_Zicsr_Zifencei with KV32-specific cache-management, PMA, and stack-guard extensions
Pipeline	5-stage in-order (Fetch, Decode, Execute, Memory, Writeback) with ID-stage branch prediction (BTB+BHT) and Return Address Stack (RAS)
Instruction Cache	Configurable size (default 4KB), 2-way set-associative, 32B line; WRAP burst fill; PMA X-bit control
Data Cache	Configurable size (default 4KB), 2-way set-associative, 32B line; write-back / write-through; write-allocate / no-allocate; dirty eviction; CMO support; PMA C-bit control
Physical Memory Attributes (PMA)	8 runtime-configurable regions; address modes OFF/TOR/NA4/NAPOT; per-region I-cache (X) and D-cache © enable; lock bit

Feature	Specification
Privilege Modes	Machine (M-mode) only
Bus Protocol	AXI4-Lite (peripherals), AXI4 (memory)
Clock Frequency	100 MHz (default, parameterizable)
Memory	2MB external RAM, supports up to 512MB address space
Peripherals	UART, SPI, I2C, DMA (4 channels), GPIO (4×32-pin banks), Timer/PWM (4 channels), CLINT, PLIC, Magic device
Interrupts	Timer, software, external (PLIC-routed)
Performance	~2.3 CPI typical; up to ~99% I-cache hit rate; D-cache provides ~1-cycle load latency on hit (workload dependent)
Implementation	RTL simulation (Verilator), FPGA synthesis supported

1.3. Document Organization

This datasheet is organized as follows:

- **Section 2**: System architecture overview with block diagram
- **Section 3**: Memory map and address decode
- **Section 4**: Processor core specification
- **Section 5**: Instruction cache specification
- **Section 6**: Data cache specification
- **Section 7**: Peripheral specifications (UART, SPI, I2C, DMA, GPIO, Timer/PWM, CLINT, PLIC, Magic device)
- **Section 8**: AXI interconnect and bus protocol
- **Section 9**: Electrical and timing characteristics
- **Section 10**: Programming guide
- **Appendices**: Register quick reference, interrupt vectors, pin descriptions

Chapter 2. System Architecture

2.1. Block Diagram

```
                    +-----------------------------------------------------+
                    |           RV32IMAC SoC Top Level                    |
                    |                                                     |
  +-----------------+-------------------------------------------------+   |
  |  KV32 Core      |                                                 |   |
  |  +------------+ |                                                 |   |
  |  |   IF       |-+->Instruction                                    |   |
  |  |   (Fetch)  |<+--  Buffer                                       |   |
  |  +-----+------+ |   (IB_DEPTH)                                    |   |
  |        |        |        |                                        |   |
  |  +-----v------+ |        v                                        |   |
  |  |   ID       | |   +----------+         +-------------+          |   |
  |  |  (Decode)  | |   | I-Cache  |<------->|  Mem Arbiter|          |   |
  |  +-----+------+ |   | (4KB/2W) |         |  (I+D paths)|          |   |
  |        |        |   +----------+         +------+------+          |   |
  |  +-----v------+ |                               |                 |   |
  |  |   EX       | |                               v                 |   |
  |  | (Execute)  | |                        +-------------+          |   |
  |  +-----+------+ |                        | AXI4-Lite   |          |   |
  |        |        |                        |  Crossbar   |          |   |
  |  +-----v------+ |   +----------+         |  (1-to-11)  |          |   |
  |  |   MEM      |-+-->| D-Cache  |<------->+------+------+          |   |
  |  |  (Memory)  |<+---| (4KB/2W) |                |                 |   |
  |  +-----+------+ |   |write-back|                |                 |   |
  |        | Store  |   +----------+                |                 |   |
  |  +-----v------+ |   Buffer(SB)                  |                 |   |
  |  |   WB       | |   +---------------------------+-----------------+---+
  |  | (Writeback)| |   | Slave 0: RAM (2MB)        |                 |   |
  |  +------------+ |   | 0x8000_0000-0x801F_FFFF   v                 |   |
  |                 |   |                    +-------------+          |   |
  |                 |   |                    |  AXI4 RAM   |          |   |
  +-----------------+   |                    |  (2MB ext)  |          |   |
                        |                    +-------------+          |   |
                        |                                             |   |
                        | Slave 1: Magic    0x4000_0000-0x4000_FFFF   |   |
                        |   +--------------------------+              |   |
                        |   | Simulation console I/O   |              |   |
                        |   | Exit code control        |              |   |
                        |   +--------------------------+              |   |
                        |                                             |   |
                        | Slave 2: CLINT    0x0200_0000-0x020B_FFFF   |   |
                        |   +--------------------------+              |   |
                        |   | Timer (mtime/mtimecmp)   |              |   |
                        |   | Software IRQ (msip)      |---->timer_irq|   |
                        |   +--------------------------+   +>sw_irq   |   |
                        |                                             |   |
                        | Slave 3: PLIC     0x0C00_0000-0x0CFF_FFFF   |   |
                        |   +--------------------------+              |   |
                        |   | Priority/Enable/Claim    |<---uart_irq  |   |
                        |   | Threshold/Complete       |<---spi_irq   |   |
                        |   +--------------+-----------+<---i2c_irq   |   |
```

```
                        |                  |            <---dma_irq   |   |
                        |                  +-------------->ext_irq    |   |
                        |                                           |   |
                        | Slave 4: DMA      0x2000_0000-0x2000_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | 4 channels, 1D/2D/3D/SG  |<-->(AXI4     |   |
                        |   | 32-bit data width        |    master    |   |
                        |   | Performance counters     |    port)     |   |
                        |   +-------------------------+             |   |
                        |                                           |   |
                        | Slave 5: UART     0x2001_0000-0x2001_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | TX/RX FIFOs (16-deep)    |<---uart_rx   |   |
                        |   | Baud rate: 25MHz max     |--->uart_tx   |   |
                        |   +-------------------------+             |   |
                        |                                           |   |
                        | Slave 6: I2C      0x2002_0000-0x2002_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | Master mode (100k/400k)  |<-->i2c_sda   |   |
                        |   | TX/RX FIFOs (8-deep)     |<-->i2c_scl   |   |
                        |   +-------------------------+             |   |
                        |                                           |   |
                        | Slave 7: SPI      0x2003_0000-0x2003_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | Master mode (4 CS lines) |--->spi_sclk  |   |
                        |   | TX/RX FIFOs (8-deep)     |--->spi_mosi  |   |
                        |   | Modes 0-3 (CPOL/CPHA)    |<---spi_miso  |   |
                        |   +-------------------------+--->spi_cs_n  |   |
                        |                                           |   |
                        | Slave 8: Timer    0x2004_0000-0x2004_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | 4 ch, 32-bit up-counter  +-->pwm[3:0]   |   |
                        |   | PWM + dual compare       +-->timer_irq  |   |
                        |   +-------------------------+             |   |
                        |                                           |   |
                        | Slave 9: GPIO     0x2005_0000-0x2005_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | 4 banks x 32 pins        |<-->gpio[127:0]|  |
                        |   | Dir/IRQ/edge/level       +-->gpio_irq   |   |
                        |   +-------------------------+             |   |
                        |                                           |   |
                        | Slave 10: WDT     0x2006_0000-0x2006_FFFF   |   |
                        |   +-------------------------+             |   |
                        |   | 32-bit countdown timer   +-->wdt_reset_o|   |
                        |   | IRQ or hw-reset on expiry+-->wdt_irq    |   |
                        |   +-------------------------+             |   |
                        +---------------------------------------------+   |
                                                                          |
    External Pins:                                                        |
      clk, rst_n                                                          |
      uart_tx, uart_rx                                                    |
      spi_sclk, spi_mosi, spi_miso, spi_cs_n[3:0]                         |
      i2c_scl_o, i2c_scl_i, i2c_scl_oe                                    |
      i2c_sda_o, i2c_sda_i, i2c_sda_oe                                    |
      gpio_out[127:0], gpio_in[127:0], gpio_oe[127:0]                     |
      pwm_out[3:0]                                                        |
      m_axi_* (external RAM interface)                                    |
```

```
                                                    |
```

2.2. System Components

2.2.1. Processor Core (kv32_core)

The RV32IMAC processor core implements a 5-stage pipeline with the following key features:

- **Instruction Fetch (IF)**: Fetches instructions from I-cache or memory via instruction buffer
- **Instruction Decode (ID)**: Decodes instructions, reads register file
- **Execute (EX)**: Executes ALU operations, computes addresses, evaluates branches
- **Memory (MEM)**: Accesses data memory via store buffer
- **Writeback (WB)**: Writes results to register file, retires instructions

The core supports:

- Data forwarding to resolve RAW hazards
- Pipeline stalls for load-use dependencies
- Dynamic branch prediction: ID-stage BTB+BHT with optional RAS-assisted return prediction
- Configurable multiply/divide (fast combinatorial or slow serial)
- Instruction buffer (IB_DEPTH=4) for outstanding instruction fetches
- Store buffer (SB_DEPTH=2) for non-blocking stores

2.2.2. Instruction Cache (kv32_icache)

An optional configurable instruction cache improves performance by reducing memory latency:

- **Size**: 4KB default (parameterizable)
- **Associativity**: 2-way set-associative (parameterizable)
- **Line Size**: 32 bytes (KV32 SoC instantiation default; the `kv32_icache` IP module default is 64 bytes)
- **Replacement Policy**: Round-robin per set
- **Bypass Mode**: Cacheable or direct-mapped via CMO operations; PMA X-bit per region
- **AXI Interface**: WRAP bursts for cache-line fills (critical-word-first)

2.2.3. Data Cache (kv32_dcache)

An optional configurable data cache reduces load/store latency:

- **Size**: 4KB default (parameterizable)
- **Associativity**: 2-way set-associative (parameterizable)
- **Line Size**: 32 bytes (8 words)
- **Write Policy**: Write-back (default) or write-through
- **Allocate Policy**: Write-allocate on store miss (default) or no-allocate (bypass)
- **Replacement Policy**: Round-robin per set
- **Dirty Eviction**: Automatically write-back dirty line before fill (write-back mode)
- **AXI Interface**: WRAP bursts for cache-line fills; AW/W/B channels for eviction and write-through
- **CMO Support**: INVAL, FLUSH, CLEAN, FLUSH_ALL operations
- **PMA Integration**: Per-region C bit controls D-cacheability; non-cacheable accesses bypass the cache

2.2.4. Physical Memory Attributes (PMA)

The PMA subsystem provides per-region cacheability control at runtime:

- **Regions**: 8 independently configurable regions
- **Address Modes**: OFF (disabled), TOR (top-of-range), NA4 (4B natural align), NAPOT (power-of-two)
- **Per-region Attributes**: X bit (I-cacheable), C bit (D-cacheable), B bit (bufferable), L bit (locked)
- **CSRs**: `pmacfg0` (0x7C0), `pmacfg1` (0x7C1), `pmaaddr0`–`pmaaddr7` (0x7C4–0x7CB)
- **Fallback Rule**: When no region matches, `addr[31]=1` → cacheable (RAM); `addr[31]=0` → non-cacheable (I/O)

2.2.5. Memory Subsystem

- **External RAM**: 2MB AXI4 SRAM at 0x8000_0000, supports burst reads/writes
- **Instruction Arbiter**: Arbitrates between instruction fetch and data memory paths
- **AXI Crossbar**: 1-to-11 interconnect routes transactions to appropriate slaves

2.2.6. Peripheral Subsystem

Seven peripheral slaves hang off the AXI crossbar:

- **UART** (`axi_uart.sv`): 8N1 serial, 16-deep TX/RX FIFOs, programmable baud rate
- **I2C** (`axi_i2c.sv`): Master-mode, 100k/400k, 8-deep TX/RX FIFOs
- **SPI** (`axi_spi.sv`): Master-mode, SPI modes 0-3, 4 CS lines, 8-deep TX/RX FIFOs
- **DMA** (`axi_dma.sv`): 4 channels, 1D/2D/3D/SG transfers, performance counters
- **GPIO** (`axi_gpio.sv`): 4 banks × 32 pins, direction/IRQ/edge/level config, loopback
- **Timer** (`axi_timer.sv`): 4 channels, 32-bit up-counter, dual compare, PWM output
- **Magic**: Simulation-only console I/O and exit-code control

2.2.7. Interrupt System

- **CLINT**: Machine-mode timer (mtime/mtimecmp) and software interrupts (msip)
- **PLIC**: Routes up to 31 external interrupt sources (UART, SPI, I2C, DMA, GPIO, Timer, WDT) with programmable priorities (0-15) and a configurable claim/complete handshake
- **Core Integration**: timer_irq, software_irq, external_irq inputs to core CSR unit

2.2.8. Power Manager (kv32_pm)

The `kv32_pm` module implements clock gating for the processor core, enabling low-power idle when the core executes a `WFI` (Wait-For-Interrupt) instruction.

2.2.8.1. Operation

When the processor executes `WFI`, `kv32_core` asserts `core_sleep_o` after all outstanding instruction-fetch and store-buffer traffic drains. `kv32_pm` then gates the core clock until any M-mode interrupt becomes active:

```
clk_en = !sleep_req_i       // core not in WFI
       | timer_irq_i         // CLINT machine-timer
       | external_irq_i      // PLIC external IRQ
       | software_irq_i      // CLINT software IRQ
       | irq_wake_pending;   // captured single-cycle pulse
```

2.2.8.2. IRQ Pulse Capture

Interrupt sources may assert for only a single cycle on the ungated system clock. To prevent the core from remaining asleep after a missed short pulse, a flip-flop clocked by the ungated `clk_i`

captures any IRQ edge and holds `clk_en=1` (`irq_wake_pending`) until the core de-asserts `sleep_req_i`. This guarantees the gated clock delivers at least one rising edge to the core's interrupt-sampling logic.

2.2.8.3. Technology Mapping

Table 1. Clock-Gate Implementation

Target	Implementation
FPGA (Xilinx UltraScale+)	`BUFGCE` primitive with `CE_TYPE="SYNC"`. Vivado infers correct setup/hold constraints on CE. CE active-high; not inverted.
Simulation / ASIC	Behavioural ICG cell: transparent latch (open on low phase of `clk_i`) followed by AND gate. Recognised by synthesis tools as a standard ICG pattern.

2.2.8.4. Timing

- **Sleep entry**: `core_sleep_o` asserted after all outstanding AXI transactions drain
- **Wake latency**: 1 clock period from IRQ assertion to first gated rising edge
- **Clock domain boundary**: CLINT and PLIC run on the ungated system clock and continue counting while the core is asleep

Chapter 3. Memory Map

3.1. Address Space Overview

The RV32IMAC SoC implements a 32-bit address space (4GB) with the following allocation:

Table 2. Complete Memory Map

Address Range	Device	Size	Description
0x0200_0000 - 0x020B_FFFF	CLINT	768KB	(only 64KB used, rest reserved per RISC-V spec)
0x0C00_0000 - 0x0CFF_FFFF	PLIC	64MB	(actual registers much smaller, per RISC-V spec)
0x2000_0000 - 0x2000_FFFF	DMA	64KB	DMA controller (4 channels)
0x2001_0000 - 0x2001_FFFF	UART	64KB	Serial communication peripheral
0x2002_0000 - 0x2002_FFFF	I2C	64KB	I2C master controller
0x2003_0000 - 0x2003_FFFF	SPI	64KB	SPI master controller
0x2004_0000 - 0x2004_FFFF	Timer	64KB	Timer/PWM controller (4 channels, 32-bit)
0x2005_0000 - 0x2005_FFFF	GPIO	64KB	General-purpose I/O controller (4 banks × 32 pins)
0x2006_0000 - 0x2006_FFFF	WDT	64KB	Hardware watchdog timer (IRQ or hardware reset on timeout)
0x4000_0000 - 0x4000_FFFF	Magic Device	64KB	Simulation control and console
0x8000_0000 - 0x801F_FFFF	RAM	2MB	Main system memory (SRAM)

3.2. Address Decode Rules

The AXI crossbar (`axi_xbar`) decodes addresses based on the high-order bits:

- **Slave 0 (RAM)**: `addr[31:21] == 11'h400` (0x8000_0000)
- **Slave 1 (Magic)**: `addr[31:16] == 16'h4000` (0x4000_0000)
- **Slave 2 (CLINT)**: `addr[31:20] == 12'h020` (0x0200_0000)
- **Slave 3 (PLIC)**: `addr[31:24] == 8'h0C` (0x0C00_0000)
- **Slave 4 (DMA)**: `addr[31:16] == 16'h2000` (0x2000_0000)
- **Slave 5 (UART)**: `addr[31:16] == 16'h2001` (0x2001_0000)
- **Slave 6 (I2C)**: `addr[31:16] == 16'h2002` (0x2002_0000)
- **Slave 7 (SPI)**: `addr[31:16] == 16'h2003` (0x2003_0000)
- **Slave 8 (Timer)**: `addr[31:16] == 16'h2004` (0x2004_0000)
- **Slave 9 (GPIO)**: `addr[31:16] == 16'h2005` (0x2005_0000)
- **Slave 10 (WDT)**: `addr[31:16] == 16'h2006` (0x2006_0000)

Unmapped addresses return an AXI SLVERR (2'b10) response on the read channel and OKAY (2'b00) on the write channel (write is silently ignored).

Chapter 4. Processor Core Specification

4.1. RV32IMAC ISA Support

The processor core implements the following RISC-V instruction set extensions:

Table 3. Supported ISA Extensions

Extension	Description
RV32I	Base integer instruction set (32-bit)
M	Integer multiplication and division
A	Atomic instructions (LR/SC, AMO)
C (Zca)	16-bit compressed instructions; expanded to 32-bit by `kv32_rvc` before the decode stage
Zicsr	CSR access instructions (CSRRW, CSRRS, CSRRC, etc.)
Zifencei	Instruction-fetch fence (FENCE.I)
Zicbom-style CMO operations	Cache-management operations used by the KV32 D-cache subsystem; see Appendix D for portability notes
KV32 stack-guard extension (non-standard)	Custom M-mode CSRs `0x7CC` (`sguard_base`) and `0x7CD` (`spmin`) plus custom exception cause `16` for stack overflow

The core supports Machine mode (M-mode) only. User mode (U-mode) and Supervisor mode (S-mode) are not implemented.

4.2. Pipeline Architecture

The core uses a 5-stage in-order pipeline:

Table 4. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Request instruction from I-cache/memory via instruction buffer
ID	Instruction Decode	Decode opcode, read source registers, generate control signals

Stage	Name	Function
EX	Execute	Execute ALU operation, compute memory address, evaluate branch condition
MEM	Memory Access	Perform load/store via data memory interface and store buffer
WB	Writeback	Write result to destination register, retire instruction

4.2.1. Instruction Buffer (IB)

The instruction buffer decouples instruction fetch from pipeline execution:

- **Depth**: IB_DEPTH (default 4, parameterizable)
- **Purpose**: Allow multiple outstanding instruction fetches to hide memory latency
- **Benefit**: Keeps pipeline fed even during I-cache misses

4.2.2. Store Buffer (SB)

The store buffer allows stores to complete asynchronously:

- **Depth**: SB_DEPTH (default 2, parameterizable)
- **Purpose**: Buffer store requests so pipeline doesn't stall on store completion
- **Ordering**: Maintains program order for stores

4.2.3. Branch Prediction and Return Address Stack

The core implements configurable branch prediction in the ID stage:

- **BTB (Branch Target Buffer)**: Direct-mapped target cache with metadata (`is_uncond`, `is_return`)
- **BHT (Branch History Table)**: 2-bit saturating counters for conditional direction prediction
- **RAS (Return Address Stack)**: Optional return-target predictor for JALR return flows

Prediction and correction behavior:

- **Prediction stage**: ID stage (`pc_id`) to avoid RVC halfword-alignment ambiguity at IF fetch-word granularity

- **Early redirect**: `bp_if_flush` redirects IF using predicted target
- **EX validation**: EX compares predicted-vs-actual outcome/target (`bp_correct`)
- **Recovery**: `need_redirect` flushes IF/ID and redirects on misprediction or unpredicted taken branches

Configuration parameters:

Parameter	Default	Description
BP_EN	1	Enable branch prediction logic
BTB_SIZE	32	BTB entry count (direct-mapped)
BHT_SIZE	64	BHT entry count (2-bit counters)
RAS_EN	1	Enable return address stack
RAS_DEPTH	8	RAS stack depth

4.2.4. Forwarding and Hazards

- **Data Forwarding**: EX→EX, MEM→EX, WB→EX paths resolve RAW hazards
- **Load-Use Stall**: Pipeline stalls 1 cycle when load result is immediately used
- **Branch Prediction**: ID-stage BTB+BHT (+ optional RAS). Correctly predicted taken branches typically incur ~1 lost slot; misprediction/unpredicted taken redirects flush IF/ID.
- **Structural Hazards**: Single memory port shared between I-fetch and D-access via arbiter

4.3. Performance Characteristics

Typical performance metrics (workload-dependent):

Table 5. Performance Metrics

Metric	Typical Value
CPI (Cycles Per Instruction)	~2.3 (I-cache only); ~1.5–1.8 with both I-cache and D-cache enabled
I-Cache Hit Rate	~99% (depends on working set size)
D-Cache Hit Rate	~95–99% (depends on access pattern and working set)
D-Cache Hit Latency	1 cycle (registered output from SRAM)

Metric	Typical Value
Correctly Predicted Taken Branch Penalty	~1 slot (ID-stage redirect model, workload/pipeline-state dependent)
Branch Misprediction Penalty	2 cycles (IF/ID flush and redirect on correction path)
Load-Use Penalty	1 cycle stall
Multiply Latency	1 cycle (FAST_MUL=1), 32 cycles (FAST_MUL=0)
Divide Latency	1 cycle (FAST_DIV=1), 33 cycles (FAST_DIV=0)

4.4. CSR (Control and Status Registers)

The core implements Machine-mode CSRs per RISC-V Privileged Spec v1.11:

Table 6. Implemented CSRs

Address	Name	Access	Description
0x300	mstatus	R/W	Machine status register
0x301	misa	R/W	ISA and extensions (reports RV32IMAC; misa=0x40001105: A=bit0, C=bit2, I=bit8, M=bit12)
0x304	mie	R/W	Machine interrupt-enable register
0x305	mtvec	R/W	Machine trap-handler base address
0x340	mscratch	R/W	Machine scratch register
0x341	mepc	R/W	Machine exception program counter
0x342	mcause	R/W	Machine trap cause
0x343	mtval	R/W	Machine trap value
0x344	mip	R/W	Machine interrupt-pending register
0xB00	mcycle	R/W	Machine cycle counter (lower 32 bits)
0xB80	mcycleh	R/W (W1C on bit 2)	Machine cycle counter (upper 32 bits)

Address	Name	Access	Description
[2]: STOP done (sticky, write 1 to clear)	minstret	R/W	Machine instructions-retired counter (lower 32 bits)
0xB82	minstreth	R/W	Machine instructions-retired counter (upper 32 bits)
0xF11	mvendorid	R/O	Vendor ID (returns 0)
0xF12	marchid	R/O	Architecture ID (returns 0)
0xF13	mimpid	R/O	Implementation ID (returns 0)
0xF14	mhartid	R/O	Hardware thread ID (returns 0)
0x7C0	pmacfg0	R/W	PMA configuration for regions 0–3 (8-bit cfg per region: `[7]=L [4:3]=A [2]=X [1]=C [0]=B`)
0x7C1	pmacfg1	R/W	PMA configuration for regions 4–7 (same format as pmacfg0)
0x7C4	pmaaddr0	R/W	PMA address register for region 0 (bits[33:2] of address, NAPOT-encoded)
0x7C5–0x7CB	pmaaddr1–pmaaddr7	R/W	PMA address registers for regions 1–7 (same format as pmaaddr0)
0x7CC	sguard_base	R/W	Stack-guard lower bound. `0` disables checking; writes to `x2`/SP below this bound raise exception cause `16`.
0x7CD	spmin	R/W	Minimum observed SP (low-water mark). Software may write to reset/reseed tracking.

4.5. Exception and Interrupt Handling

4.5.1. Exception Types

Table 7. Exception Causes (mcause[31]=0)

Code	Name	Description
0	Instruction address misaligned	PC not 4-byte aligned (should not occur in normal operation)
1	Instruction access fault	I-fetch AXI error response
2	Illegal instruction	Undefined opcode or malformed instruction
3	Breakpoint	EBREAK instruction executed
4	Load address misaligned	Load address not naturally aligned
5	Load access fault	Load AXI error response
6	Store/AMO address misaligned	Store address not naturally aligned
7	Store/AMO access fault	Store AXI error response
11	Environment call from M-mode	ECALL instruction executed in M-mode
16	Stack overflow (KV32 custom)	SP written below `sguard_base` while stack guard is enabled

4.5.2. Interrupt Types

Table 8. Interrupt Causes (mcause[31]=1)

Code	Name	Source
3	Machine software interrupt	CLINT MSIP register
7	Machine timer interrupt	CLINT mtime >= mtimecmp
11	Machine external interrupt	PLIC aggregated external interrupt

4.5.3. Trap Handling

On exception or interrupt:

1. PC is saved to `mepc`
2. Trap cause is written to `mcause`
3. Trap value (e.g., faulting address) is written to `mtval`
4. `mstatus.MIE` is cleared (disable interrupts)

5. PC is set to `mtvec` (trap vector address)

Return from trap: `MRET` instruction restores `mepc` to PC and restores interrupt enable.

Chapter 5. Instruction Cache Specification

5.1. Overview

The instruction cache (kv32_icache) reduces instruction fetch latency by caching frequently accessed code:

Table 9. I-Cache Configuration

Parameter	Default Value
CACHE_SIZE	4096 bytes (4KB)
CACHE_LINE_SIZE	32 bytes (8 words)
CACHE_WAYS	2 (2-way set-associative)
Number of Sets	64 (computed: SIZE / (LINE_SIZE × WAYS))
Replacement Policy	Round-robin (per-set victim pointer)
Write Policy	N/A (instruction cache is read-only)

5.2. Cache Organization

- **Address Decomposition**:
  - Tag: bits [31:11] (21 bits)
  - Index: bits [10:5] (6 bits, selects 1 of 64 sets)
  - Word Offset: bits [4:2] (3 bits, selects 1 of 8 words)
  - Byte Offset: bits [1:0] (always 00 for aligned word access)
- **Storage**:
  - Valid bits: flip-flops (2 ways × 64 sets = 128 bits)
  - Tag storage: SRAM macros (2 instances, 64 entries × 21 bits)
  - Data storage: SRAM macros (2 instances, 512 entries × 32 bits)

5.3. Cache Operations

5.3.1. Normal Fetch (Cache Hit)

1. Core asserts `imem_req_valid` with address

2. Cache performs lookup (1 cycle)
3. On hit, returns data via `imem_resp_data` next cycle
4. Core accepts response and continues

Latency: 1 cycle (registered output)

5.3.2. Cache Miss

1. Cache detects miss in lookup stage
2. Issues AXI WRAP burst for entire cache line (8 beats)
3. Returns critical word first (early restart) to unblock core
4. Remaining beats drain in background while core continues
5. Victim way is selected via round-robin pointer

Latency: Memory latency + 1 cycle (critical-word-first optimization)

5.3.3. Bypass Mode

When cache is disabled via CMO (Cache Management Operation):

- Every fetch issues single-beat AXI INCR transaction
- No caching occurs, data is forwarded directly to core
- Useful for debugging or non-cacheable code regions

5.4. Cache Management Operations (CMO)

The cache supports software-initiated management via the `cmo_*` sideband interface:

Table 10. Supported CMO Operations

Opcode	Name	Description
2’b00	CMO_INVAL	Invalidate specific cache line matching address
2’b01	CMO_DISABLE	Disable cache, enter bypass mode
2’b10	CMO_ENABLE	Re-enable cache
2’b11	CMO_FLUSH_ALL	Invalidate all cache entries (FENCE.I support)

CMO operations are typically triggered by `FENCE.I` or `cbo.inval` instructions in

> the core's MEM stage.

5.5. Performance Counters (Simulation Only)

The cache provides performance monitoring signals (not synthesized):

- `perf_req_cnt`: Total fetch requests
- `perf_hit_cnt`: Cache hits
- `perf_miss_cnt`: Cache misses
- `perf_bypass_cnt`: Bypass fetches (cache disabled)
- `perf_fill_cnt`: Completed line fills
- `perf_cmo_cnt`: CMO operations executed

Chapter 6. Data Cache Specification

6.1. Overview

The data cache (`kv32_dcache`) reduces load/store latency by caching frequently accessed data. It supports both write-back and write-through policies with write-allocate or no-allocate behaviour on store misses.

Table 11. D-Cache Configuration

Parameter	Default Value
DCACHE_SIZE	4096 bytes (4KB)
DCACHE_LINE_SIZE	32 bytes (8 words)
DCACHE_WAYS	2 (2-way set-associative)
Number of Sets	64 (computed: SIZE / (LINE_SIZE × WAYS))
Replacement Policy	Round-robin (per-set victim pointer)
Write Policy	Write-back (DCACHE_WRITE_BACK=1); write-through with DCACHE_WRITE_BACK=0
Allocate Policy	Write-allocate (DCACHE_WRITE_ALLOC=1); no-allocate with DCACHE_WRITE_ALLOC=0

6.2. Cache Organisation

- **Address Decomposition**:
  - Tag: bits [31:11] (21 bits)
  - Index: bits [10:5] (6 bits, selects 1 of 64 sets)
  - Word Offset: bits [4:2] (3 bits, selects 1 of 8 words)
  - Byte Offset: bits [1:0] (byte within word)
- **Storage**:
  - Valid bits: flip-flops (2 ways × 64 sets = 128 bits)
  - Dirty bits: flip-flops (2 ways × 64 sets = 128 bits, write-back mode only)
  - Tag storage: SRAM macros (2 instances, 64 entries × 21 bits)
  - Data storage: SRAM macros (2 instances, 512 entries × 32 bits, byte-enable write)

6.3. Cache Operations

6.3.1. Load Hit

1. Core asserts `core_req_valid` with address and `mem_read=1`
2. Cache performs lookup (1 cycle, SRAM read launched speculatively in `S_IDLE`)
3. On hit, returns data via `core_resp_data` next cycle (zero-stall if core is ready)

Latency: 1 cycle (registered output)

6.3.2. Load Miss — Clean Victim

1. Cache detects miss in lookup stage
2. Issues AXI WRAP burst for entire cache line (8 beats, critical-word-first)
3. Returns critical word early to unblock core
4. Remaining beats drain in background

Latency: Memory latency + 1 cycle (critical-word-first optimisation)

6.3.3. Load Miss — Dirty Victim (Write-Back Mode)

1. Cache detects miss; victim line is dirty
2. Writes dirty line back to memory (AXI write: AW → W × 8 → B)
3. Fetches new line with AXI WRAP burst
4. Returns critical word to core

Latency: Memory latency × 2 (eviction + fill round-trip)

6.3.4. Store Hit

1. Cache detects hit; data SRAM performs read-modify-write in `S_HIT_WR`
2. For write-back: dirty bit set; no AXI traffic
3. For write-through: dirty bit unchanged; AXI write issued (`S_WT_AW → S_WT_W → S_WT_B`)

Write-Back Latency: 1 cycle (no AXI traffic) **Write-Through Latency**: Memory latency

6.3.5. Store Miss

- **Write-Allocate** (`DCACHE_WRITE_ALLOC=1`): Allocate new line (evict dirty if needed), fill from memory, then perform hit-store.
- **No-Allocate** (`DCACHE_WRITE_ALLOC=0`): Write directly to memory via AXI — no line allocation.

6.3.6. PMA Bypass (Non-Cacheable Access)

When `use_dcache=0` (PMA C bit clear or CMO_DISABLE):

- Reads: single-beat AXI INCR read, data forwarded directly to core
- Writes: single-beat AXI INCR write with byte strobes

6.4. Cache Management Operations (CMO)

The cache supports software-initiated management via the `cmo_*` sideband interface:

Table 12. Supported CMO Operations

Opcode	Name	Description
2'b00	CMO_INVAL	Invalidate specific cache line matching address (clears valid bit; no writeback)
2'b01	CMO_FLUSH	Flush (write-back + invalidate) specific cache line matching address
2'b10	CMO_CLEAN	Write back dirty line without invalidating (write-back mode only)
2'b11	CMO_FLUSH_ALL	Write back all dirty lines; invalidate all entries (DMA pre/post-transfer use)

`CMO_FLUSH_ALL` is triggered by `FENCE.I` and is required before DMA transfers to maintain coherence.

6.5. Performance Counters (Simulation Only)

The cache provides performance monitoring signals (not synthesized):

- `perf_req_cnt`: Total data accesses
- `perf_hit_cnt`: Cache hits (load or store)

- `perf_miss_cnt`: Cache misses
- `perf_bypass_cnt`: Non-cacheable bypasses (PMA or CMO_DISABLE)
- `perf_fill_cnt`: Completed line fills
- `perf_evict_cnt`: Dirty line evictions
- `perf_cmo_cnt`: CMO operations executed

Chapter 7. Peripheral Specifications

7.1. UART (Universal Asynchronous Receiver/Transmitter)

7.1.1. Overview

The UART peripheral provides serial communication with configurable baud rate and 8N1 format (8 data bits, no parity, 1 stop bit).

Table 13. UART Features

Feature	Specification
Base Address	0x2001_0000
Size	64KB (only first 0x18 bytes used)
Data Format	8 bits, no parity, 1 stop bit (8N1)
Baud Rate	Configurable (default 25 Mbaud, min 4 clocks/bit)
TX FIFO Depth	16 entries
RX FIFO Depth	16 entries
Interrupt	PLIC source ID 1

7.1.2. Register Map

Table 14. UART Registers

Offset	Name	Access	Description
0x00	DATA	R/W	**Read**: Pop byte from RX FIFO **Write**: Push byte to TX FIFO
0x04	STATUS	R/O	**[0]**: tx_busy (1=TX FIFO full, can't write) **[1]**: tx_full (same as bit 0) **[2]**: rx_ready (1=RX FIFO not empty, can read) **[3]**: rx_full (1=RX FIFO full)
0x08	IE (Interrupt Enable)	R/W	**[0]**: Enable RX not-empty interrupt **[1]**: Enable TX empty interrupt
0x0C	IS (Interrupt Status)	R/O	**[0]**: RX not-empty (level) **[1]**: TX empty (level)

Offset	Name	Access	Description
0x10	LEVEL	R/O	**[3:0]**: RX FIFO count **[11:8]**: TX FIFO count
0x14	CTRL	R/W	**[0]**: Loopback enable (1=TX→RX internal loopback)

Interrupt asserts when `(IE & IS) != 0`.

7.1.3. Baud Rate Configuration

The UART baud rate is determined by the SoC parameter `BAUD_RATE`:

```
CLKS_PER_BIT = CLK_FREQ / BAUD_RATE
```

Hardware Constraint: `CLKS_PER_BIT >= 4` (absolute minimum for RX sampling).

Recommended: `CLKS_PER_BIT >= 8` for production use (provides timing margin).

Table 15. Common Baud Rate Examples (100 MHz Clock)

Baud Rate	CLKS_PER_BIT	Notes
115200	868	Standard slow baud, large margin
921600	109	Fast baud, good margin
1 Mbaud	100	Round number, good margin
3.125 Mbaud	32	Fast, acceptable margin
12.5 Mbaud	8	Very fast, minimum recommended
25 Mbaud	4	Maximum (hardware minimum, lab use only)

Baud rates requiring `CLKS_PER_BIT < 4` will not work correctly due to insufficient sampling resolution in the RX state machine.

7.1.4. Interrupt Behavior

- **RX Interrupt**: Fires when RX FIFO transitions from empty to non-empty
- **TX Interrupt**: Fires when TX FIFO becomes empty (all data transmitted)

Software should:

- **RX ISR**: Read all available bytes from DATA register until STATUS[2]=0
- **TX ISR**: Refill TX FIFO by writing to DATA register

7.1.5. External Pins

Table 16. UART Pins

Signal	Direction	Description
uart_tx	Output	UART transmit data (to external RX pin)
uart_rx	Input	UART receive data (from external TX pin)

Both signals are asynchronous to system clock (standard UART timing).

7.2. SPI (Serial Peripheral Interface)

7.2.1. Overview

The SPI peripheral provides master-mode serial communication for interfacing with SPI slaves (flash, ADC, sensors, etc.).

Table 17. SPI Features

Feature	Specification
Base Address	0x2003_0000
Size	64KB (only first 0x1C bytes used)
Mode	Master only
Clock Modes	Modes 0-3 (CPOL/CPHA configurable)
Data Width	8 bits per transfer
Clock Frequency	Programmable divider: `SCLK = CLK / (2 * (DIV + 1))`

Feature	Specification
TX FIFO Depth	8 entries
RX FIFO Depth	8 entries
Chip Selects	4 (active-low)
Interrupt	PLIC source ID 2

7.2.2. Register Map

Table 18. SPI Registers

Offset	Name	Access	Description
0x00	CTRL	R/W	**[0]**: Enable (1=enable SPI controller) **[1]**: CPOL (clock polarity: 0=idle low, 1=idle high) **[2]**: CPHA (clock phase: 0=leading edge, 1=trailing edge) **[3]**: Loopback (1=MOSI→MISO internal loopback) **[7:4]**: CS chip select (active-low, 4 bits)
0x04	DIV	R/W	Clock divider: `SCLK = CLK / (2 * (DIV + 1))` DIV=0 → CLK/2 (fastest), DIV=255 → CLK/512
0x08	TX (TX Data)	W/O	Write: push byte to TX FIFO (triggers transfer) Read: returns 0
0x0C	RX (RX Data)	R/O	Read: pop byte from RX FIFO Write: ignored
0x10	STATUS	R/O	**[0]**: Busy (1=transfer in progress) **[1]**: TX ready (!tx_full, can accept data) **[2]**: RX valid (!rx_empty, data available) **[3]**: TX empty (TX FIFO drained) **[4]**: RX full (RX FIFO full)
0x14	IE (Interrupt Enable)	R/W	**[0]**: Enable RX not-empty interrupt **[1]**: Enable TX empty interrupt
0x18	IS (Interrupt Status)	R/O	**[0]**: RX not-empty (level) **[1]**: TX empty (level)

7.2.3. SPI Modes

Table 19. SPI Clock Modes

Mode	CPOL	CPHA	Description
0	0	0	Clock idle low, sample on leading (rising) edge
1	0	1	Clock idle low, sample on trailing (falling) edge
2	1	0	Clock idle high, sample on leading (falling) edge
3	1	1	Clock idle high, sample on trailing (rising) edge

7.2.4. Clock Divider Examples (100 MHz System Clock)

Table 20. Common SPI Clock Frequencies

DIV Value	SCLK Frequency	Application
0	50 MHz	Maximum speed (may exceed slave limits)
1	33.33 MHz	Very fast SPI (high-speed flash)
3	25 MHz	Fast SPI (typical maximum for many devices)
9	10 MHz	Standard SPI
24	4 MHz	Conservative SPI
49	2 MHz	Slow devices (SD card init)
99	1 MHz	Very slow devices

7.2.5. Usage Flow

1. Configure CTRL[7:4] to select chip (active-low: clear bit to select)
2. Set CTRL[2:1] for desired CPOL/CPHA mode
3. Set DIV for desired SCLK frequency
4. Set CTRL[0]=1 to enable controller
5. Write data to TX register to initiate transfer
6. Poll STATUS[2] or wait for RX interrupt
7. Read received data from RX register

8. Repeat steps 5-7 for each byte
9. Set CTRL[7:4]=0xF to deselect all chips (all CS high)

7.2.6. External Pins

Table 21. SPI Pins

Signal	Direction	Description
spi_sclk	Output	SPI serial clock
spi_mosi	Output	Master Out, Slave In (data to slave)
spi_miso	Input	Master In, Slave Out (data from slave)
spi_cs_n[3:0]	Output	Chip select lines (active-low)

7.3. I2C (Inter-Integrated Circuit)

7.3.1. Overview

The I2C peripheral provides master-mode I2C communication for interfacing with sensors, EEPROMs, and other I2C devices.

Table 22. I2C Features

Feature	Specification
Base Address	0x2002_0000
Size	64KB (only first 0x1C bytes used)
Mode	Master only
Speed	Standard (100 kHz) and Fast (400 kHz)
Addressing	7-bit address mode
TX FIFO Depth	8 entries
RX FIFO Depth	8 entries
Interrupt	PLIC source ID 3

7.3.2. Register Map

Table 23. I2C Registers

Offset	Name	Access	Description
0x00	CTRL	R/W	**[0]**: Enable (1=enable I2C controller) **[1]**: Start (write 1 to issue START condition) **[2]**: Stop (write 1 to issue STOP condition) **[3]**: Read (1=read byte from slave, 0=write byte) **[4]**: NACK (1=send NACK after read, 0=send ACK)
0x04	DIV	R/W	Clock divider: `SCL_period = CLK / (4 * (DIV + 1))` For 100kHz: DIV = (CLK_FREQ / 400000) - 1 For 400kHz: DIV = (CLK_FREQ / 1600000) - 1
0x08	TX (TX Data)	W/O	Write: push byte to TX FIFO Read: returns 0
0x0C	RX (RX Data)	R/O	Read: pop byte from RX FIFO Write: ignored
0x10	STATUS	R/O	**[0]**: Busy (1=transfer in progress) **[1]**: TX ready (!tx_full, can accept data) **[2]**: RX valid (!rx_empty, data available) **[3]**: ACK received (1=slave ACKed last byte)
0x14	IE (Interrupt Enable)	R/W	**[0]**: Enable RX not-empty interrupt **[1]**: Enable TX empty interrupt **[2]**: Enable STOP done interrupt
0x18	IS (Interrupt Status)	R/O	**[0]**: RX not-empty (level) **[1]**: TX empty (level) **[2]**: STOP done (level)

7.3.3. Clock Divider Examples (100 MHz System Clock)

Table 24. Common I2C Speeds

Speed	DIV Value	Formula
100 kHz (Standard)	249	(100M / 400k) - 1
400 kHz (Fast)	62	(100M / 1.6M) - 1
1 MHz (Fast-mode Plus)	24	(100M / 4M) - 1

The I2C bus frequency formula accounts for 4 clock cycles per SCL period (rise/fall/hold times in

the state machine).

7.3.4. Usage Flow

7.3.4.1. Write Transaction

1. Write slave address (shifted left, bit 0=0 for write) to TX register
2. Set CTRL[1]=1 to issue START
3. Wait for STATUS[1]=1 (TX ready) or TX interrupt
4. Write data bytes to TX register
5. Check STATUS[3]=1 to verify slave ACK
6. Set CTRL[2]=1 to issue STOP
7. Wait for IS[2]=1 (STOP done interrupt), then acknowledge by writing 1 to IS[2]

7.3.4.2. Read Transaction

1. Write slave address (shifted left, bit 0=1 for read) to TX register
2. Set CTRL[1]=1 to issue START
3. Set CTRL[3]=1 (read mode)
4. For each byte:
    - Wait for STATUS[2]=1 or RX interrupt
    - Read byte from RX register
    - For last byte, set CTRL[4]=1 (NACK)
5. Set CTRL[2]=1 to issue STOP

7.3.5. External Pins

Table 25. I2C Pins

Signal	Direction	Description
i2c_scl_o	Output	SCL output data (0=pull low)
i2c_scl_i	Input	SCL input data (for clock stretching)
i2c_scl_oe	Output	SCL output enable (1=high-Z, 0=drive)
i2c_sda_o	Output	SDA output data (0=pull low)

Signal	Direction	Description
i2c_sda_i	Input	SDA input data
i2c_sda_oe	Output	SDA output enable (1=high-Z, 0=drive)

I2C uses open-drain signaling. External pull-up resistors (typically 4.7kΩ) are required on both SCL and SDA lines.

7.4. DMA (Direct Memory Access Controller)

7.4.1. Overview

The DMA controller provides high-performance memory-to-memory and peripheral-to-memory transfers with minimal CPU intervention.

Table 26. DMA Features

Feature	Specification
Base Address	0x2000_0000
Size	64KB
Channels	4 independent channels
Transfer Modes	1D (linear), 2D (strided), 3D (planar), Scatter-Gather
Data Width	32 bits (parameterizable to 64/128)
Max Burst Length	16 beats (parameterizable)
FIFO Depth	16 entries per channel (parameterizable)
Interrupt	PLIC source ID 4 (aggregated, all channels)

7.4.2. Register Map

Each channel has registers at offset `BASE + channel * 0x40`:

*Table 27. Per-Channel Registers (offset from BASE + ch*0x40)*

Offset	Name	Access	Description
0x00	CTRL	R/W	**[0]**: EN (enable channel) **[1]**: START (write 1 to arm transfer, auto-clears) **[2]**: STOP (write 1 to abort, auto-clears) **[4:3]**: MODE (00=1D, 01=2D, 10=3D, 11=SG) **[5]**: SRC_INC (1=increment source address) **[6]**: DST_INC (1=increment destination address) **[7]**: IE (interrupt enable for this channel)
0x04	STAT	R/W1C	**[0]**: BUSY (transfer in progress, read-only) **[1]**: DONE (transfer complete, write 1 to clear) **[2]**: ERR (AXI error, write 1 to clear)
0x08	SRC_ADDR	R/W	Source base address (must be 4-byte aligned)
0x0C	DST_ADDR	R/W	Destination base address (must be 4-byte aligned)
0x10	XFER_CNT	R/W	Transfer size in bytes (1D: total; 2D/3D: bytes per row)
0x14	SRC_STRIDE	R/W	Source stride in bytes between rows (2D/3D only)
0x18	DST_STRIDE	R/W	Destination stride in bytes between rows (2D/3D only)
0x1C	ROW_CNT	R/W	Number of rows (2D/3D only)
0x20	SRC_PLANE_STRIDE	R/W	Source stride in bytes between planes (3D only)
0x24	DST_PLANE_STRIDE	R/W	Destination stride in bytes between planes (3D only)
0x28	PLANE_CNT	R/W	Number of planes (3D only)
0x2C	SG_ADDR	R/W	Scatter-gather descriptor list base address (SG mode)
0x30	SG_CNT	R/W	Number of scatter-gather descriptors (SG mode)

Table 28. Global Registers (offset from BASE)

Offset	Name	Access	Description
0xF00	IRQ_STAT	R/W1C	Channel interrupt status (bit N = channel N done/err, write 1 to clear)
0xF04	IRQ_EN	R/W	Per-channel IRQ global enable (bit N = channel N)

Offset	Name	Access	Description
0xF08	DMA_ID	R/O	DMA controller ID register (reads 0xD4A0_0100)
0xF10	PERF_CTRL	R/W	**[0]**: Enable performance counters **[1]**: Write 1 to reset all counters
0xF14	PERF_CYCLES	R/O	Total cycles elapsed while PERF_CTRL[0]=1
0xF18	PERF_RD_BYTES	R/O	Total bytes read via AXI (cumulative)
0xF1C	PERF_WR_BYTES	R/O	Total bytes written via AXI (cumulative)

7.4.3. Transfer Modes

7.4.3.1. 1D Transfer (Linear)

Simple contiguous memory copy:

- Set `SRC_ADDR`, `DST_ADDR`, `XFER_CNT`
- Set `CTRL[4:3]=00` (MODE=1D)
- Set `CTRL[5:6]` for address increment behavior
- Set `CTRL[1]=1` to start

7.4.3.2. 2D Transfer (Strided)

Copy 2D arrays with row stride:

- Set `SRC_ADDR`, `DST_ADDR`, `XFER_CNT` (bytes per row)
- Set `SRC_STRIDE`, `DST_STRIDE` (bytes between row starts)
- Set `ROW_CNT` (number of rows)
- Set `CTRL[4:3]=01` (MODE=2D)

Example: Copy 16×16 pixel subregion from 640×480 framebuffer:

```
XFER_CNT = 16 * 4        // 16 pixels x 4 bytes/pixel
SRC_STRIDE = 640 * 4     // Source pitch
DST_STRIDE = 16 * 4      // Destination pitch (packed)
ROW_CNT = 16             // 16 rows
```

7.4.3.3. 3D Transfer (Planar)

Copy 3D volumes with plane stride:

- Configure as 2D, plus set `SRC_PLANE_STRIDE`, `DST_PLANE_STRIDE`, `PLANE_CNT`
- Set `CTRL[4:3]=10` (MODE=3D)

7.4.3.4. Scatter-Gather Transfer

Chain multiple transfers from descriptor list:

- Each descriptor is 16 bytes: `{src_addr, dst_addr, xfer_cnt, mode_ctrl}`
- Set `SG_ADDR` to descriptor list base address
- Set `SG_CNT` to number of descriptors
- Set `CTRL[4:3]=11` (MODE=SG)

7.4.4. Interrupt Behavior

- Each channel can raise interrupt on completion (DONE) or error (ERR)
- Channel interrupts are aggregated to single PLIC source (ID 4)
- Software reads `IRQ_STAT` to determine which channel(s) triggered
- Write 1 to `IRQ_STAT` bits to acknowledge

7.4.5. Performance Monitoring

Enable performance counters to measure DMA throughput:

```
DMA_PERF_CTRL = 0x3;    // Enable counters and reset
// ... run workload ...
cycles = DMA_PERF_CYCLES;
rd_bytes = DMA_PERF_RD_BYTES;
wr_bytes = DMA_PERF_WR_BYTES;
bandwidth_MBps = (rd_bytes + wr_bytes) / cycles * CLK_FREQ / 1e6;
```

7.4.6. AXI Interface

The DMA controller has a dedicated AXI4 master port (32-bit data width default):

- Burst optimization: DMA controller generates INCR bursts up to `MAX_BURST_LEN` beats
- Page boundary handling: Bursts never cross 4KB AXI page boundaries

- Error handling: AXI SLVERR/DECERR sets channel STAT[2]=1 and aborts transfer

7.5. GPIO (General-Purpose I/O)

7.5.1. Overview

The GPIO controller provides four independent 32-pin banks with configurable direction, atomic bit-manipulation, edge/level interrupt detection, and loopback test mode.

Table 29. GPIO Features

Feature	Specification
Base Address	0x2005_0000
Size	64KB
Banks	4 independent banks (banks 0–3)
Pins Per Bank	32 (128 total I/O pins)
Output Control	Write, atomic set (W1S), atomic clear (W1C)
Input	Synchronous registered input path
Interrupt	Per-pin enable, edge (rising/falling) or level (high/low), PLIC source ID 5
Loopback	Per-bank loopback enable for self-test (output routed back to input)
Capability Register	`{VERSION[31:16], NUM_BANKS[15:8], NUM_PINS[7:0]}` = `0x0001_0120`

7.5.2. Register Map

All groups repeat for banks 0–3 (bank *b* register at `BASE + group_offset + b * 4`):

Table 30. GPIO Register Map

Base Offset	Name	Access	Description
0x00–0x0C	DATA_OUT[3:0]	R/W	Output data register for each bank
0x10–0x1C	SET[3:0]	WO	Atomic set: write-1 bits set corresponding DATA_OUT bits

Base Offset	Name	Access	Description
0x20–0x2C	CLEAR[3:0]	WO	Atomic clear: write-1 bits clear corresponding DATA_OUT bits
0x30–0x3C	DATA_IN[3:0]	RO	Current synchronised input pin state
0x40–0x4C	DIR[3:0]	R/W	Direction: 1 = output, 0 = input (default 0)
0x50–0x5C	IE[3:0]	R/W	Interrupt enable mask per pin
0x60–0x6C	TRIGGER[3:0]	R/W	Trigger mode: 1 = edge-sensitive, 0 = level-sensitive
0x70–0x7C	POLARITY[3:0]	R/W	Polarity: edge — 1 = rising, 0 = falling; level — 1 = high, 0 = low
0x80–0x8C	IS[3:0]	W1C	Interrupt status: write-1-to-clear per pin
0x90–0x9C	LOOPBACK[3:0]	R/W	Loopback enable: 1 = route output to input path (for testing)
0xA0	CAPABILITY	RO	`{VERSION[31:16], NUM_BANKS[15:8], NUM_PINS[7:0]}`

7.5.3. Interrupt Behavior

- Any enabled pin whose interrupt condition is met asserts `gpio_irq` to PLIC (source 5)
- Interrupt status register IS[b] shows which pins fired (one bit per pin per bank)
- Write-1-to-clear to IS[b] after servicing; IRQ de-asserts once all IS bits are cleared
- TRIGGER and POLARITY can be set independently per pin

7.5.4. External Pins

Table 31. GPIO External Pins

Pin	Direction	Description
gpio_out[127:0]	Output	Driven from DATA_OUT when DIR=1

Pin	Direction	Description
gpio_in[127:0]	Input	Sampled to DATA_IN; unused when DIR=1
gpio_oe[127:0]	Output	Output-enable for tri-state buffers (equals DIR)

7.6. Timer/PWM Controller

7.6.1. Overview

The Timer provides four independent 32-bit up-counters with PWM output, prescaler divider, dual compare registers for mid-period and period-end interrupts, and per-channel enable.

Table 32. Timer Features

Feature	Specification
Base Address	0x2004_0000
Size	64KB
Channels	4 independent channels
Counter Width	32 bits
Prescaler	16-bit value in CTRL[31:16] — clock divided by (PRESCALE + 1)
Compare Registers	COMPARE1: mid-period IRQ or PWM falling edge COMPARE2: period end (counter auto-reloads to 0) and IRQ
PWM	Active-high or active-low (via CTRL[4]); duty set by COMPARE1/COMPARE2 ratio
Interrupt	Per-channel interrupt enable (CTRL[3]); individual IRQ per channel to PLIC sources 6–9 (KV_PLIC_SRC_TIMER0–KV_PLIC_SRC_TIMER3)
Capability Register	`{VERSION[31:16], NUM_CHANNELS[15:8], COUNTER_WIDTH[7:0]}` = `0x0001_0420`

7.6.2. Register Map

Per-channel registers at `BASE + channel * 0x20` (channels 0–3):

Table 33. Timer Per-Channel Registers (BASE + ch × 0x20)

Offset	Name	Access	Description
0x00	COUNT	R/W	Current counter value (read: current count; write: preload)
0x04	COMPARE1	R/W	Mid-period compare: triggers interrupt and/or PWM falling edge at this count
0x08	COMPARE2	R/W	Period compare: counter reloads to 0 when COUNT == COMPARE2; triggers period IRQ
0x0C	CTRL	R/W	**[0]**: EN — timer enable **[1]**: PWM_EN — PWM output enable **[3]**: INT_EN — interrupt enable **[4]**: PWM_POL — PWM polarity (1 = active high) **[31:16]**: PRESCALE — clock divider value

Global registers (shared across all channels):

Table 34. Timer Global Registers

Offset	Name	Access	Description
0x80	INT_STATUS	W1C	Interrupt status: bits [3:0] correspond to channels 3:0; write-1-to-clear
0x84	INT_ENABLE	R/W	Global interrupt enable per channel: bits [3:0]; must be set in addition to CTRL[3]
0x88	CAPABILITY	RO	`{VERSION[31:16], NUM_CHANNELS[15:8], COUNTER_WIDTH[7:0]}`

7.6.3. Timer Modes

Simple mode — periodic interrupt, period = COMPARE2 + 1 ticks:

1. Set COMPARE1 = 0, COMPARE2 = period - 1
2. Set CTRL = EN | INT_EN | PRESCALE(n)
3. On interrupt: read/clear INT_STATUS; counter auto-reloads

Dual compare mode — two events per period:

1. COMPARE1 fires a mid-period interrupt (e.g. for phase-correct PWM updates)
2. COMPARE2 fires a period-end interrupt and reloads the counter

PWM mode:

1. Set CTRL[1] = PWM_EN; output goes high at COUNT=0, low at COUNT=COMPARE1, reloads at COMPARE2
2. Duty cycle = COMPARE1 / (COMPARE2 + 1)
3. Update COMPARE1 on-the-fly to adjust duty without stopping the timer

7.6.4. Timing Example (100 MHz clock, 1 kHz interrupt)

```
PRESCALE = 0         // No prescaling; clock = 100 MHz
COMPARE2 = 99999     // Period = 100000 ticks = 1 ms
INT_ENABLE = 0x1     // Enable channel 0 global IRQ
CTRL = EN | INT_EN   // Start timer with interrupt
```

For a 1 kHz 50% duty-cycle PWM:

```
COMPARE1 = 49999     // Falling edge at 50% duty
COMPARE2 = 99999     // Period = 1 ms
CTRL = EN | PWM_EN | PWM_POL | INT_EN
```

7.6.5. Interrupt Behavior

- Each channel's interrupt is enabled by both CTRL[3] (channel) and INT_ENABLE[ch] (global)
- INT_STATUS bit is set on COMPARE1 match (if dual mode) or COMPARE2 match
- Write-1-to-clear INT_STATUS[ch] in ISR; timer continues running
- Each channel has an independent PLIC source: channel N maps to PLIC source `KV_PLIC_SRC_TIMER0 + N` (sources 6–9)

7.6.6. External Pins

Table 35. Timer External Pins

Pin	Direction	Description
pwm_out[3:0]	Output	PWM output per channel (active when CTRL[1]=1)

7.7. Watchdog Timer (WDT)

7.7.1. Overview

The WDT provides a hardware safety net that resets the system (or raises an IRQ) if software fails to periodically service it.

Table 36. WDT Features

Feature	Specification
Base Address	0x2006_0000
Size	64KB
PLIC Source	10 (`KV_PLIC_SRC_WDT`) — only when INTR_EN=1
External output	`wdt_reset_o` — asserts and latches when INTR_EN=0 and expiry occurs; cleared only by `rst_n`
Counter width	32-bit countdown (configurable via LOAD register)

7.7.2. Register Map

Table 37. WDT Register Map

Offset	Name	Access	Description
0x00	CTRL	R/W	[0]=EN: start/stop the countdown. [1]=INTR_EN: expiry action (1=assert PLIC IRQ, 0=assert `wdt_reset_o`)
0x04	LOAD	R/W	Reload value — written before enabling the WDT. Value loaded into COUNT on enable or KICK.
0x08	COUNT	RO	Current countdown value. Counts down from LOAD to 0.
0x0C	KICK	WO	Any write reloads COUNT from LOAD (\"pets\" the watchdog). Ignores written data.
0x10	STATUS	R/W1C	[0]=WDT_INT — set when count expires with INTR_EN=1. Write 1 to clear.
0x14	CAP	RO	Capability: `0x0001_0020` (version=1, counter width=32)

Addresses > 0x17 within the WDT aperture return SLVERR.

7.7.3. Operation

1. Write reload value to LOAD.

2. Write `0x03` to CTRL (EN=1, INTR_EN=1 for IRQ mode, or `0x01` for reset mode).
3. Periodically write any value to KICK before COUNT reaches 0.

On expiry (COUNT reaches 0)

- **IRQ mode** (INTR_EN=1): STATUS[0] is set; PLIC source 10 asserts. Software must claim, handle, kick (or clear STATUS), and complete.
- **Reset mode** (INTR_EN=0): `wdt_reset_o` asserts and *latches* until `rst_n`; software cannot recover. System integrator feeds `wdt_reset_o` into the system reset tree.

The COUNT register latches at zero after expiry; a KICK is required to resume counting.

7.7.4. Interrupt Behavior

- `wdt_irq` to PLIC = `STATUS[0] & CTRL[1]` (only fires in IRQ mode)
- `wdt_reset_o` = latched register set on expiry in reset mode; cleared only by `rst_n`
- In simulation (`kv32sim`): reset mode calls `exit(2)`; Spike plugin calls `sim_→htif_exit(2)`

7.7.5. External Pins

Table 38. WDT External Pins

Pin	Direction	Description
wdt_reset_o	Output	Hardware reset request. Latches high on reset-mode expiry. System integrator feeds into reset tree.

7.8. CLINT (Core-Local Interruptor)

7.8.1. Overview

The CLINT provides machine-mode timer and software interrupts per RISC-V Privileged Specification.

Table 39. CLINT Features

Feature	Specification
Base Address	0x0200_0000
Size	768KB (large per RISC-V spec, only 64KB used)

Feature	Specification
Functionality	64-bit real-time counter (mtime), 64-bit compare (mtimecmp), software interrupt (msip)
Timer Tick	1 per clock cycle (default), configurable in trace mode
Interrupt Outputs	timer_irq, software_irq to core

7.8.2. Register Map

Table 40. CLINT Registers

Offset	Name	Access	Description
0x0000	MSIP	R/W	Machine Software Interrupt Pending Write 1 to bit 0 to trigger software interrupt
0x4000	MTIMECMP_LO	R/W	Timer compare register (lower 32 bits)
0x4004	MTIMECMP_HI	R/W	Timer compare register (upper 32 bits)
0xBFF8	MTIME_LO	R/W	Real-time counter (lower 32 bits)
0xBFFC	MTIME_HI	R/W	Real-time counter (upper 32 bits)

Register offsets match RISC-V CLINT specification for single-hart systems.

7.8.3. Timer Operation

The 64-bit `mtime` counter increments by 1 every clock cycle. Timer interrupt fires when:

```
mtime >= mtimecmp
```

Software clears timer interrupt by writing a new (larger) value to `mtimecmp`.

Example: Set timer for 1ms from now (100 MHz clock):

```
uint64_t mtime = read_mtime();
uint64_t mtimecmp = mtime + 100000;  // 1ms = 100k cycles @ 100MHz
write_mtimecmp(mtimecmp);
```

7.8.4. Software Interrupt Operation

Write `MSIP[0] = 1` to trigger machine software interrupt immediately. Clear by writing `MSIP[0] =`

`0`.

Typical use: Inter-processor interrupt (IPI) on multi-hart systems, or self-interrupt for context switching.

7.9. PLIC (Platform-Level Interrupt Controller)

7.9.1. Overview

The PLIC routes external interrupt sources to the processor core with programmable priorities.

Table 41. PLIC Features

Feature	Specification
Base Address	0x0C00_0000
Size	16MB (0x0C00_0000–0x0CFF_FFFF, per RISC-V PLIC specification `PLIC_SIZE = 0x01000000`)
Interrupt Sources	Up to 31 (source 0 reserved)
Priority Levels	0-15 (0 = never interrupt, 1-15 = increasing priority)
Target Contexts	1 (hart 0, M-mode)
Claim/Complete Protocol	Standard RISC-V PLIC mechanism

7.9.2. Register Map

Table 42. PLIC Registers

Offset	Name	Access	Description
0x000000	PRIORITY[0]	R/O	Reserved, always reads 0
0x000004 × i	PRIORITY[i]	R/W	Priority for interrupt source i (i=1..31, 4-bit)
0x001000	PENDING	R/O	Interrupt pending bits [31:1] (bit i = source i pending)
0x002000	ENABLE	R/W	Interrupt enable bits [31:1] for context 0

Offset	Name	Access	Description
0x200000	THRESHOLD	R/W	Priority threshold for context 0 (4-bit) Interrupts fire only if source priority > threshold
0x200004	CLAIM/COMPLETE	R/W	**Read**: Claim highest-priority pending interrupt (returns source ID) **Write**: Complete interrupt (write source ID to clear)

7.9.3. Interrupt Sources

The following peripherals are connected to the PLIC:

Table 43. PLIC Interrupt Source IDs

Source ID	Peripheral	Description
1	UART	UART RX not-empty or TX empty
2	SPI	SPI RX not-empty or TX empty
3	I2C	I2C RX not-empty, TX empty, or STOP done
4	DMA	DMA channel done or error (aggregated)
5	GPIO	GPIO pin interrupt (any bank, W1C per-pin)
6	Timer0	Timer channel 0 compare match (`KV_PLIC_SRC_TIMER0`)
7	Timer1	Timer channel 1 compare match (`KV_PLIC_SRC_TIMER1`)
8	Timer2	Timer channel 2 compare match (`KV_PLIC_SRC_TIMER2`)
9	Timer3	Timer channel 3 compare match (`KV_PLIC_SRC_TIMER3`)
10	WDT	Watchdog timer expired with INTR_EN=1 (`KV_PLIC_SRC_WDT`)
11-31	Reserved	Future peripheral expansion

7.9.4. Interrupt Flow

1. **Pending**: Peripheral asserts `irq_src[i]`, PLIC sets `PENDING[i]=1`
2. **Arbitration**: PLIC selects highest-priority enabled pending interrupt where `priority[i] > threshold`

3. **Notification**: PLIC asserts `external_irq` to core
4. **Claim**: ISR reads `CLAIM` register to get source ID, `PENDING[i]` remains set
5. **Service**: ISR handles the interrupt, clears condition in peripheral
6. **Complete**: ISR writes source ID to `COMPLETE` register
7. **Clear**: If peripheral de-asserted `irq_src[i]`, PLIC clears `PENDING[i]`

> If peripheral IRQ remains asserted after complete, `PENDING[i]` is immediately reset (level-sensitive behavior).

7.9.5. Priority Arbitration

When multiple interrupts are pending:

- Highest priority wins (priority 7 > priority 1)
- On priority tie, lowest source ID wins (source 1 > source 2)

Example configuration:

```
PRIORITY[1] = 5;  // UART high priority
PRIORITY[2] = 3;  // SPI medium priority
PRIORITY[3] = 3;  // I2C medium priority
PRIORITY[4] = 1;  // DMA low priority
PRIORITY[5] = 4;  // GPIO medium-high priority
PRIORITY[6] = 2;  // Timer low-medium priority
THRESHOLD = 0;    // Allow all enabled interrupts
ENABLE = 0x7E;    // Enable sources 1-6
```

7.10. Magic Device (Simulation Only)

7.10.1. Overview

The Magic Device provides a simple interface for simulation control and console I/O without requiring a real UART.

> This peripheral is intended for simulation and testing only. It is not synthesizable for FPGA/ASIC implementation.

Table 44. Magic Device Features

Feature	Specification
Base Address	0x4000_0000
Size	64KB
Functionality	Console output, simulation exit control, and 512-byte non-cacheable memory (NCM)

7.10.2. Register Map

Table 45. Magic Device Registers

Offset	Name	Access	Description
0x0000	CONSOLE	W/O	Write character (low byte) to simulation console
0x0004	EXIT	W/O	Write exit code to terminate simulation Value 1 = PASS (exit 0), Value ≠1 = FAIL (exit code N)
0x1000-0x11FF	NCM[127:0]	R/W	Non-cacheable memory window (128 x 32-bit words, 512 B total) for uncached instruction/data access tests

7.10.3. AXI Response Behavior

`axi_magic` returns AXI response `OKAY` only for these decoded regions:

- `0x4000_0000` (`CONSOLE`, word-aligned alias)
- `0x4000_0004` (`EXIT`, word-aligned alias)
- `0x4000_1000-0x4000_11FF` (NCM window)

All other addresses in the Magic-device aperture return `SLVERR` on read/write transactions.

NCM implementation notes (from RTL):

- NCM is 128 x 32-bit words (index `addr[8:2]`)
- Reads return the stored 32-bit word
- Writes store `axi_wdata` as a full word (byte strobes are not merged per-byte)

7.10.4. Exit Code Encoding

Magic device uses HTIF (Host-Target Interface) encoding for exit codes:

```
Pass:     write 1
Fail:     write ((exit_code << 1) | 1)
```

Example:

```
volatile uint32_t *magic_exit = (uint32_t *)0x40000004;
volatile uint32_t *magic_console = (uint32_t *)0x40000000;

// Print to console
*magic_console = 'H';
*magic_console = 'i';
*magic_console = '\n';

// Exit with success
*magic_exit = 1;  // Simulation terminates with exit(0)
```

Chapter 8. AXI Interconnect and Bus Protocol

8.1. AXI4-Lite Overview

The SoC uses AXI4-Lite for peripheral communication and AXI4 for high-bandwidth memory access.

Table 46. AXI Protocol Summary

Feature	Specification
Protocol Version	AXI4 (AMBA 4 specification)
Peripheral Protocol	AXI4-Lite (32-bit address, 32-bit data)
Memory Protocol	AXI4 (32-bit address, 32-bit data, burst support)
Interconnect Topology	1-to-10 crossbar (single master, 10 slaves)
Outstanding Transactions	Supported via AXI ID mechanism (ID width = 4 bits, max 16 outstanding)
Burst Types	INCR (incrementing), WRAP (wrapping, for I-cache)
Response Codes	OKAY (00), SLVERR (10), DECERR (11)

8.2. Crossbar Architecture

The AXI crossbar (`axi_xbar`) routes transactions from the core to appropriate slave devices:

- **Master Port**: Connected to memory arbiter (combines I-fetch + D-access paths)
- **Slave Ports**: 10 devices (RAM, CLINT, PLIC, UART, I2C, SPI, DMA, GPIO, Timer, Magic)

8.2.1. Address Decode Logic

Decoding is priority-based (higher-numbered slaves checked first):

```
if (addr[31:16] == 16'hFFFF)       → Slave 9 (Magic)
else if (addr[31:16] == 16'h2005)  → Slave 8 (Timer)
else if (addr[31:16] == 16'h2004)  → Slave 7 (GPIO)
else if (addr[31:12] == 20'h2003_0) → Slave 6 (DMA)
else if (addr[31:16] == 16'h2002)  → Slave 5 (SPI)
else if (addr[31:16] == 16'h2001)  → Slave 4 (I2C)
else if (addr[31:16] == 16'h2000)  → Slave 3 (UART)
else if (addr[31:26] == 6'h03)     → Slave 2 (PLIC)
else if (addr[31:20] == 12'h020)   → Slave 1 (CLINT)
else if (addr[31:21] == 11'h400)   → Slave 0 (RAM)
```

```
else                            → SLVERR (unmapped)
```

8.2.2. Error Responses

- **Unmapped address read**: Crossbar returns `SLVERR` response with data = 0xDEADBEEF
- **Unmapped address write**: Crossbar returns `OKAY` response (write silently dropped)

8.3. AXI4 Memory Interface

The external RAM interface uses full AXI4 with burst support:

Table 47. AXI4 Memory Features

Feature	Specification
Data Width	32 bits
Burst Length	1-256 beats (ARLEN/AWLEN encoding)
Burst Types	INCR (normal access), WRAP (I-cache line fills)
Burst Size	4 bytes per beat (ARSIZE=3’b010)
Address Alignment	4-byte aligned
Write Strobes	4 bits (byte-level granularity)

8.3.1. I-Cache Burst Transactions

The instruction cache uses WRAP bursts for critical-word-first cache-line fills:

- Burst type: `ARBURST = 2'b10` (WRAP)
- Burst length: `ARLEN = (line_size / 4) - 1` (e.g., 15 for 64-byte line)
- Starting address: Critical word address (PC)
- Wrap boundary: Cache-line boundary

8.3.2. D-Cache Burst Transactions

The data cache uses WRAP bursts for cache-line fills and INCR for dirty evictions and write-through:

- **Fill (read)**: `ARBURST = 2'b10` (WRAP), `ARLEN = (line_size / 4) - 1` (e.g., 7 for 32-byte line)
- **Dirty eviction (write)**: `AWBURST = 2'b01` (INCR), `AWLEN = WORDS_PER_LINE - 1`, full cache-line

burst starting at line base address

- **Write-through (write)**: `AWBURST = 2'b01` (INCR), `AWLEN = 0` (single beat), byte strobes indicate sub-word granularity
- **Non-cacheable bypass**: `ARBURST = 2'b01` (INCR), `ARLEN = 0` (single beat) for reads; same for writes

Example: D-cache WRAP fill from address 0x80000054 with 32-byte cache line:

```
ARCHR = 0x80000054
ARLEN = 7           // 8 beats
ARBURST = WRAP
Beats returned:
  0: 0x80000054 (critical word, returned first)
  1: 0x80000058
  2: 0x8000005C
  3: 0x80000040 (wrap to line boundary)
  4: 0x80000044
  5: 0x80000048
  6: 0x8000004C
  7: 0x80000050
```

Example: I-cache WRAP fill from address 0x80000014 with 64-byte cache line:

```
ARADDR = 0x80000014
ARLEN = 15          // 16 beats
ARBURST = WRAP
Beats returned:
  0: 0x80000014 (critical word, returned first)
  1: 0x80000018
  2: 0x8000001C
  3: 0x80000020
  4: 0x80000024
  5: 0x80000028
  6: 0x8000002C
  7: 0x80000030
  8: 0x80000034
  9: 0x80000038
  A: 0x8000003C
  B: 0x80000000 (wrap to line boundary)
  C: 0x80000004
  D: 0x80000008
  E: 0x8000000C
  F: 0x80000010
```

D-Cache also uses WRAP bursts for fills; see Section 6 for the D-Cache-specific example.

8.3.3. Data Access Transactions

Data loads/stores without D-cache use single-beat INCR bursts. When D-cache is enabled, most accesses are served from cache without AXI traffic; only misses, dirty evictions, and write-through stores generate AXI transactions (see Data Cache Specification).

8.4. Bus Performance

Typical AXI transaction latencies:

Table 48. AXI Transaction Latencies

Transaction Type	Latency	Notes
Peripheral read (hit)	2-3 cycles	1 cycle AR, 1-2 cycles R
Peripheral write	1-2 cycles	1 cycle AW+W, write buffered
RAM read (no wait)	3 cycles	1 cycle AR, 2 cycles pipeline, 1 cycle R
RAM write (no wait)	2 cycles	1 cycle AW+W, 1 cycle B
I-cache line fill	10-20 cycles	8-beat WRAP burst + arbitration

Chapter 9. Electrical and Timing Characteristics

9.1. Clock and Reset

Table 49. Clock and Reset Specifications

Parameter	Value	Description
clk	Input	System clock (100 MHz nominal)
CLK_FREQ	100_000_000	Default clock frequency parameter (Hz)
rst_n	Active-low	Asynchronous reset, must be asserted for ≥2 clock cycles
Reset Recovery	2 cycles	Minimum cycles after rst_n de-assertion before operation

9.2. Timing Parameters

Table 50. Timing Characteristics (100 MHz, Tclk=10ns)

Parameter	Min	Max	Description
Setup Time (AXI signals)	2.0ns	-	Input setup before clock edge
Hold Time (AXI signals)	1.0ns	-	Input hold after clock edge
Clock-to-Output (AXI signals)	-	8.0ns	Output valid after clock edge
I-Cache Lookup Latency	1 cycle	-	Hit latency (registered output)
I-Cache Miss Penalty	10 cycles	50 cycles	Depends on memory latency
Load-to-Use Latency	2 cycles	-	Pipeline load latency
Branch Mispredict Penalty	2 cycles	-	Flush IF and ID stages (2 bubbles)
Interrupt Latency	3 cycles	10 cycles	From assertion to first trap instruction

9.3. Power Consumption (Estimated)

Power estimates are for Xilinx 7-series FPGA implementation at 100 MHz, 1.0V core voltage. Actual values depend on technology node, activity factor, and synthesis options.

Table 51. Power Consumption Estimates

Component	Power	Notes
Processor Core	~15 mW	Without I-cache, typical activity
I-Cache (4KB, 2-way)	~10 mW	SRAM power dominant
D-Cache (4KB, 2-way)	~10 mW	SRAM power dominant; dirty eviction adds ~1 mW
AXI Interconnect	~5 mW	Crossbar + arbiters
Peripherals (total)	~8 mW	UART, SPI, I2C, DMA idle
Total SoC	~50 mW	Typical workload @ 100 MHz (both caches enabled)

9.4. Environmental Specifications

Table 52. Operating Conditions (FPGA implementation)

Parameter	Value	Description
Operating Temperature	0°C to 85°C	Commercial temperature range
Core Voltage	1.0V ±5%	Xilinx 7-series VCCO
I/O Voltage	1.8V, 2.5V, 3.3V	Configurable per I/O bank

Chapter 10. Programming Guide

10.1. Boot Sequence

The processor boots from address `0x8000_0000` (RAM base) on reset:

1. Reset vector: PC = 0x8000_0000
2. Bootloader/firmware executes (typically copied to RAM by JTAG or external loader)
3. Initialize stack pointer
4. Initialize CSRs (mstatus, mtvec, mie)
5. Initialize peripherals (UART, timers, etc.)
6. Enable interrupts (clear mstatus.MIE)
7. Jump to main application

10.2. Memory Initialization

For simulation, the ELF loader (`elfloader.cpp`) loads program sections into RAM via DPI-C before simulation starts. For FPGA, use:

- **JTAG**: Load program via JTAG debugger (OpenOCD, etc.)

10.3. Interrupt Handling

10.3.1. Setting Up Interrupts

```c
#include "csr.h"
#include "kv_platform.h"
#include "kv_plic.h"
#include "kv_clint.h"

void setup_interrupts(void) {
    // Set trap vector (must be 4-byte aligned)
    write_csr(mtvec, (uint32_t)trap_entry);

    // Enable M-mode interrupts in mstatus
    set_csr(mstatus, MSTATUS_MIE);

    // Enable specific interrupt types in mie
    set_csr(mie, MIE_MEIE | MIE_MTIE | MIE_MSIE);

    // Configure PLIC
    kv_plic_set_priority(KV_PLIC_SRC_UART, 5);
    kv_plic_enable_interrupt(KV_PLIC_SRC_UART);
```

```
    kv_plic_set_threshold(0);

    // Configure CLINT timer for periodic interrupt (1ms)
    uint64_t mtime = kv_clint_read_mtime();
    kv_clint_write_mtimecmp(mtime + 100000);  // +1ms @ 100MHz
}
```

10.3.2. Trap Handler Example

```
void trap_handler(void) {
    uint32_t mcause = read_csr(mcause);
    uint32_t mepc = read_csr(mepc);

    if (mcause & 0x80000000) {
        // Interrupt
        uint32_t irq_code = mcause & 0x1F;
        switch (irq_code) {
        case 7:  // Timer interrupt
            handle_timer_interrupt();
            break;
        case 11: // External interrupt (PLIC)
            handle_external_interrupt();
            break;
        case 3:  // Software interrupt
            handle_software_interrupt();
            break;
        }
    } else {
        // Exception
        handle_exception(mcause, mepc);
    }
}

void handle_timer_interrupt(void) {
    // Advance mtimecmp for next interrupt
    uint64_t mtimecmp = kv_clint_read_mtimecmp();
    kv_clint_write_mtimecmp(mtimecmp + 100000);  // +1ms
}

void handle_external_interrupt(void) {
    // Claim interrupt from PLIC
    uint32_t source = kv_plic_claim_interrupt();

    switch (source) {
    case KV_PLIC_SRC_UART:
        uart_isr();
        break;
    case KV_PLIC_SRC_SPI:
        spi_isr();
        break;
    // ... handle other sources ...
    }

    // Complete interrupt
    kv_plic_complete_interrupt(source);
```

```
}
```

10.4. Peripheral Usage Examples

10.4.1. UART Echo Program

```
#include "kv_uart.h"

void uart_echo(void) {
    kv_uart_init(115200);  // Configure baud rate

    while (1) {
        if (kv_uart_rx_ready()) {
            uint8_t c = kv_uart_getc();
            kv_uart_putc(c);  // Echo character
        }
    }
}
```

10.4.2. SPI Flash Read

```
#include "kv_spi.h"

#define SPI_CMD_READ 0x03

void spi_read_flash(uint32_t addr, uint8_t *buf, size_t len) {
    kv_spi_init(1000000, KV_SPI_MODE0);  // 1 MHz, Mode 0

    kv_spi_select(0);  // Assert CS[0]

    // Send READ command + 24-bit address
    kv_spi_transfer(SPI_CMD_READ);
    kv_spi_transfer((addr >> 16) & 0xFF);
    kv_spi_transfer((addr >> 8) & 0xFF);
    kv_spi_transfer(addr & 0xFF);

    // Read data
    for (size_t i = 0; i < len; i++) {
        buf[i] = kv_spi_transfer(0x00);
    }

    kv_spi_deselect(0);  // De-assert CS[0]
}
```

10.4.3. I2C EEPROM Write

```
#include "kv_i2c.h"

#define EEPROM_ADDR 0x50  // 7-bit I2C address
```

```c
void i2c_eeprom_write(uint8_t mem_addr, uint8_t data) {
    kv_i2c_init(100000);  // 100 kHz

    kv_i2c_start();
    kv_i2c_write(EEPROM_ADDR << 1);  // Address + W bit
    kv_i2c_write(mem_addr);           // Memory address
    kv_i2c_write(data);               // Data byte
    kv_i2c_stop();

    // Wait for write cycle to complete (typical 5ms)
    for (volatile int i = 0; i < 500000; i++);
}
```

10.4.4. DMA Memory Copy

```c
#include "kv_dma.h"

void dma_memcpy(void *dst, const void *src, size_t len) {
    // Configure channel 0 for 1D transfer
    kv_dma_channel_config(0, (uint32_t)src, (uint32_t)dst, len,
                          KV_DMA_CTRL_MODE_1D |
                          KV_DMA_CTRL_SRC_INC |
                          KV_DMA_CTRL_DST_INC);

    // Enable interrupt
    kv_dma_channel_enable_interrupt(0);

    // Start transfer
    kv_dma_channel_start(0);

    // Wait for completion (or use interrupt)
    while (kv_dma_channel_busy(0));

    // Check for errors
    if (kv_dma_channel_error(0)) {
        // Handle error
    }
}
```

10.5. Performance Optimization Tips

1. **Enable I-Cache**: Set `ICACHE_EN=1` parameter for significant speedup (~2-5× depending on workload)
2. **Enable D-Cache**: Set `DCACHE_EN=1` for data-access speedup on load/store-intensive workloads (~1-cycle load latency on hit vs. 3+ cycles to RAM)
3. **Tune Cache Write Policy**: Use `DCACHE_WRITE_BACK=1` (default) for best performance; use `DCACHE_WRITE_BACK=0` (write-through) where strong write ordering is required
4. **Increase Buffer Depths**: Larger `IB_DEPTH` and `SB_DEPTH` improve throughput in memory-

intensive code

5. **Use DMA for Bulk Transfers**: Offload large memory copies to DMA controller; issue `cbo.flush` / `FENCE.I` before/after DMA for coherence
6. **Minimize Load-Use Dependencies**: Reorder instructions to avoid pipeline stalls
7. **Use Fast Multiply/Divide**: Set `FAST_MUL=1` and `FAST_DIV=1` for single-cycle operations
8. **Align Data Structures**: 4-byte alignment avoids misaligned access penalties
9. **Batch Peripheral I/O**: Use FIFOs to buffer multiple bytes before checking status

10.6. Debugging

10.6.1. Simulation Debug

Use the magic device for quick debugging:

```
#define debug_printf(fmt, ...) \
    do { \
        char buf[128]; \
        snprintf(buf, sizeof(buf), fmt, ##__VA_ARGS__); \
        for (char *p = buf; *p; p++) \
            KV_MAGIC_CONSOLE = *p; \
    } while(0)

debug_printf("Checkpoint: pc=0x%08x, x10=%d\n", read_csr(mepc), x10);
```

10.6.2. Waveform Analysis

Generate VCD waveform dumps for RTL simulation:

```
make WAVE=1 rtl-simple
gtkwave build/tb_kv32_soc.vcd
```

Key signals to observe:

- **Pipeline stages**: `kv32_core.pc_if`, `state_id`, `alu_result_ex`
- **Memory transactions**: `imem_req_valid`, `dmem_resp_valid`, `axi_xbar` signals
- **Interrupts**: `timer_irq`, `external_irq`, `trap_entry`

10.6.3. Instruction Trace Comparison

Compare RTL and software simulator traces to debug mismatches:

```
make compare-simple
diff sim_trace.txt rtl_trace.txt
```

The trace format is:

```
<cycle> <pc> <instruction> <disassembly>
```

Chapter 11. Conclusion

This datasheet provides a complete reference for the RV32IMAC SoC, covering architecture, memory map, peripheral specifications, electrical characteristics, and programming guidelines. For additional information, refer to:

- **RISC-V Specifications**: https://riscv.org/technical/specifications/
- **AMBA AXI4 Specification**: https://developer.arm.com/documentation/ihi0022/latest/
- **Project Repository**: https://github.com/YourOrg/kv32 (example)

For questions or support, contact: support@example.com

Appendix A: Register Quick Reference

A.1. CLINT Registers

Address	Name	Access	Reset Value
0x0200_0000	MSIP	R/W	0x0000_0000
0x0200_4000	MTIMECMP_LO	R/W	0x4000_FFFF
0x0200_4004	MTIMECMP_HI	R/W	0x4000_FFFF
0x0200_BFF8	MTIME_LO	R/W	0x0000_0000
0x0200_BFFC	MTIME_HI	R/W	0x0000_0000

A.2. PLIC Registers

Address	Name	Access	Reset Value
0x0C00_0000	PRIORITY[0]	R/O	0x0000_0000
0x0C00_0004-0x0C00_007C	PRIORITY[1..31]	R/W	0x0000_0000
0x0C00_1000	PENDING	R/O	0x0000_0000
0x0C00_2000	ENABLE	R/W	0x0000_0000
0x0C20_0000	THRESHOLD	R/W	0x0000_0000
0x0C20_0004	CLAIM/COMPLETE	R/W	0x0000_0000

A.3. UART Registers

Address	Name	Access	Reset Value
0x2001_0000	DATA	R/W	0x0000_0000
0x2001_0004	STATUS	R/O	0x0000_0002
0x2001_0008	IE	R/W	0x0000_0000
0x2001_000C	IS	R/O	0x0000_0000
0x2001_0010	LEVEL	R/O	0x0000_0000

Address	Name	Access	Reset Value
0x2001_0014	CTRL	R/W	0x0000_0000
0x2001_0018	CAP	R/O	`{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}`

A.4. SPI Registers

Address	Name	Access	Reset Value
0x2003_0000	CTRL	R/W	0x0000_00F0
0x2003_0004	DIV	R/W	0x0000_0000
0x2003_0008	TX	W/O	-
0x2003_000C	RX	R/O	0x0000_0000
0x2003_0010	STATUS	R/O	0x0000_000A
0x2003_0014	IE	R/W	0x0000_0000
0x2003_0018	IS	R/O	0x0000_0000
0x2003_001C	CAP	R/O	`{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}`

A.5. I2C Registers

Address	Name	Access	Reset Value
0x2002_0000	CTRL	R/W	0x0000_0000
0x2002_0004	DIV	R/W	0x0000_0000
0x2002_0008	TX	W/O	-
0x2002_000C	RX	R/O	0x0000_0000
0x2002_0010	STATUS	R/O	0x0000_0002
0x2002_0014	IE	R/W	0x0000_0000
0x2002_0018	IS	R/W (W1C on bit 2)	0x0000_0000

Address	Name	Access	Reset Value
0x2002_001C	CAP	R/O	`{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}`

A.6. DMA Global Registers

Address	Name	Access	Reset Value
0x2000_0F00	IRQ_STAT	R/W1C	0x0000_0000
0x2000_0F04	IRQ_EN	R/W	0x0000_0000
0x2000_0F08	DMA_ID	R/O	0xD4A0_0100
0x2000_0F0C	CAPABILITY	R/O	`{VERSION[31:16], NUM_CHANNELS[15:8], MAX_BURST_LEN[7:0]}`
0x2000_0F10	PERF_CTRL	R/W	0x0000_0000
0x2000_0F14	PERF_CYCLES	R/O	0x0000_0000
0x2000_0F18	PERF_RD_BYTES	R/O	0x0000_0000
0x2000_0F1C	PERF_WR_BYTES	R/O	0x0000_0000

A.7. GPIO Registers (Bank 0 shown; bank *b* at `BASE + group + bx4`)

Address	Name	Access	Reset Value
0x2005_0000	DATA_OUT0	R/W	0x0000_0000
0x2005_0010	SET0	W/O	-
0x2005_0020	CLEAR0	W/O	-
0x2005_0030	DATA_IN0	R/O	(pin state)
0x2005_0040	DIR0	R/W	0x0000_0000
0x2005_0050	IE0	R/W	0x0000_0000
0x2005_0060	TRIGGER0	R/W	0x0000_0000
0x2005_0070	POLARITY0	R/W	0x0000_0000
0x2005_0080	IS0	W1C	0x0000_0000

Address	Name	Access	Reset Value
0x2005_0090	LOOPBACK0	R/W	0x0000_0000
0x2005_00A0	CAPABILITY	R/O	`0x0001_0120`

A.8. Timer Registers (Channel 0 shown; channel *n* at `BASE + n×0x20`)

Address	Name	Access	Reset Value
0x2004_0000	COUNT0	R/W	0x0000_0000
0x2004_0004	COMPARE1_0	R/W	0x0000_0000
0x2004_0008	COMPARE2_0	R/W	0x4000_FFFF
0x2004_000C	CTRL0	R/W	0x0000_0000
0x2004_0080	INT_STATUS	W1C	0x0000_0000
0x2004_0084	INT_ENABLE	R/W	0x0000_0000
0x2004_0088	CAPABILITY	R/O	`0x0001_0420`

Appendix B: Interrupt Vector Table

Table 53. Machine-Mode Interrupt Vectors

Code	Type	Description	Source
0	Exception	Instruction address misaligned	Core
1	Exception	Instruction access fault	I-fetch AXI error
2	Exception	Illegal instruction	Core
3	Exception	Breakpoint	EBREAK
4	Exception	Load address misaligned	Core
5	Exception	Load access fault	Load AXI error
6	Exception	Store/AMO address misaligned	Core
7	Exception	Store/AMO access fault	Store AXI error
11	Exception	Environment call from M-mode	ECALL
16	Exception	Stack overflow (KV32 custom)	SP < `sguard_base`
0x8000_0003	Interrupt	Machine software interrupt	CLINT MSIP
0x8000_0007	Interrupt	Machine timer interrupt	CLINT timer
0x8000_000B	Interrupt	Machine external interrupt	PLIC

mcause[31]=1 indicates interrupt; mcause[30:0] contains interrupt code.

Appendix C: Pin Descriptions

C.1. Clock and Reset

Signal	I/O	Width	Description
clk	Input	1	System clock (100 MHz nominal)
rst_n	Input	1	Asynchronous active-low reset

C.2. UART Interface

Signal	I/O	Width	Description
uart_tx	Output	1	UART transmit data
uart_rx	Input	1	UART receive data

C.3. SPI Interface

Signal	I/O	Width	Description
spi_sclk	Output	1	SPI serial clock
spi_mosi	Output	1	SPI Master Out, Slave In
spi_miso	Input	1	SPI Master In, Slave Out
spi_cs_n	Output	4	SPI chip select (active-low)

C.4. I2C Interface

Signal	I/O	Width	Description
i2c_scl_o	Output	1	I2C clock output (0=pull low)
i2c_scl_i	Input	1	I2C clock input (for stretching)
i2c_scl_oe	Output	1	I2C clock output enable (1=high-Z)
i2c_sda_o	Output	1	I2C data output (0=pull low)
i2c_sda_i	Input	1	I2C data input
i2c_sda_oe	Output	1	I2C data output enable (1=high-Z)

I2C requires external pull-up resistors (4.7kΩ typical) on SCL and SDA.

C.5. GPIO Interface

Signal	I/O	Width	Description
gpio_out	Output	128	Pin output data (driven when DIR=1 per pin)
gpio_in	Input	128	Pin input state (captured synchronously)
gpio_oe	Output	128	Output enable per pin for tri-state buffers (equals DIR)

C.6. Timer / PWM Interface

Signal	I/O	Width	Description
pwm_out	Output	4	PWM output per timer channel (active when CTRL[1]=1)

Signal	I/O	Width	Description
m_axi_awaddr	Output	32	Write address
m_axi_awlen	Output	8	Write burst length (beats - 1)
m_axi_awburst	Output	2	Write burst type (INCR=01)
m_axi_awvalid	Output	1	Write address valid
m_axi_awready	Input	1	Write address ready
m_axi_wdata	Output	32	Write data
m_axi_wstrb	Output	4	Write byte strobes
m_axi_wlast	Output	1	Write last beat
m_axi_wvalid	Output	1	Write data valid
m_axi_wready	Input	1	Write data ready
m_axi_bresp	Input	2	Write response (OKAY=00)

Signal	I/O	Width	Description
m_axi_bvalid	Input	1	Write response valid
m_axi_bready	Output	1	Write response ready
m_axi_araddr	Output	32	Read address
m_axi_arlen	Output	8	Read burst length (beats - 1)
m_axi_arsize	Output	3	Read burst size (010=4 bytes)
m_axi_arburst	Output	2	Read burst type (WRAP=10, INCR=01)
m_axi_arvalid	Output	1	Read address valid
m_axi_arready	Input	1	Read address ready
m_axi_rdata	Input	32	Read data
m_axi_rresp	Input	2	Read response (OKAY=00)
m_axi_rlast	Input	1	Read last beat
m_axi_rvalid	Input	1	Read data valid
m_axi_rready	Output	1	Read data ready

Appendix D: RISC-V Architectural Extensions and KV32-Specific Features

D.1. Scope

This appendix summarizes the processor from the **architecture** point of view:

- Which standard RISC-V ISA extensions are implemented
- Which standard privileged-architecture features are present
- Which KV32 features are architecture-visible but non-standard
- What software portability assumptions are safe to make

This section is intentionally architecture-focused. Device register maps for DMA, UART, SPI, I2C, GPIO, Timer, WDT, CLINT, PLIC, and the Magic device are platform details, not ISA extensions.

D.2. ISA and Privileged Architecture Summary

Item	Status	Architectural Meaning
RV32I	Implemented	Base 32-bit integer instruction set.
M	Implemented	Integer multiply/divide instructions.
A	Implemented	Atomic memory operations (`LR/SC`, AMOs).
C	Implemented	16-bit compressed instructions; expanded before the main decode path.
Zicsr	Implemented	CSR access instructions used for machine-mode control/status operations.
Zifencei	Implemented	`FENCE.I` supported for instruction-stream synchronization.
Zicbom-style CMO operations	Implemented by KV32 cache subsystem	Cache block maintenance operations are exposed for D-cache management.
Privilege modes	Machine mode only	No S-mode, U-mode, virtual memory, or hypervisor support.

D.3. Standard Architectural Features Implemented

D.3.1. Machine-Mode Trap Architecture

KV32 implements the standard machine-mode trap CSRs and trap return flow:

- `mstatus`
- `mie`
- `mtvec`
- `mscratch`
- `mepc`
- `mcause`
- `mtval`
- `mip`

Supported architectural trap classes:

- Instruction address misaligned
- Instruction access fault
- Illegal instruction
- Breakpoint
- Load address misaligned
- Load access fault
- Store/AMO address misaligned
- Store/AMO access fault
- Environment call from M-mode
- Stack overflow (custom cause 16)
- Machine software interrupt
- Machine timer interrupt
- Machine external interrupt

Trap entry uses direct `mtvec` mode and return uses `MRET`.

D.3.2. Standard Counters

The core implements machine and unprivileged counter aliases for:

- `mcycle` / `mcycleh`
- `minstret` / `minstreth`
- `cycle` / `cycleh`
- `instret` / `instreth`
- `time` / `timeh`

D.3.3. Atomic Extension Scope

The `A` extension support covers the standard RV32A architectural model:

- `LR.W` / `SC.W`
- Word-sized AMOs

No 64-bit atomic operations are implemented because the core is RV32.

D.3.4. Compressed Instruction Support

Compressed instructions are supported architecturally via the `C` extension. Internally, KV32 expands 16-bit instructions into an internal 32-bit representation before decode. This is a micro-architectural implementation detail; architecturally, software observes standard RVC behavior.

D.4. Standard Optional Extensions with KV32-Specific Behavior

D.4.1. `FENCE.I` (`Zifencei`)

`FENCE.I` is a standard RISC-V instruction and is architecturally supported. On KV32, the cache subsystem may additionally perform a full D-cache write-back so that self-modifying code becomes visible to subsequent instruction fetches. Software may rely on the standard `FENCE.I` semantics, but this extra write-back behavior is KV32-specific and must not be assumed on unrelated RISC-V implementations.

D.4.2. Cache Block Management (`Zicbom`-style)

KV32 exposes cache maintenance operations corresponding to cache-block invalidate, clean, and flush behavior. These operations are used by the D-cache subsystem and software cache management helpers.

Architectural interpretation:

- They are not part of the RV32IMAC base profile.
- Their availability must be treated as optional by portable software.
- Their exact interaction with KV32 PMA cacheability settings is implementation-specific.

D.5. KV32-Specific Non-Standard Architectural Features

D.5.1. Runtime-Configurable PMA CSRs

KV32 implements a custom set of M-mode CSRs for Physical Memory Attributes (PMA):

- `pmacfg0` at `0x7C0`
- `pmacfg1` at `0x7C1`
- `pmaaddr0`-`pmaaddr7` at `0x7C4`-`0x7CB`

These CSRs are **not** part of the standard RISC-V privileged architecture.

Architectural role:

- Define 8 programmable address regions
- Support `OFF`, `TOR`, `NA4`, and `NAPOT` matching modes
- Control per-region I-cacheability (`X`), D-cacheability (`C`), and bufferability (`B`)
- Provide a lock bit (`L`) to freeze configuration until reset

The address matching model is intentionally PMP-like, but the PMA CSR block itself is KV32-specific.

D.5.2. Stack-Guard CSRs and Stack-Overflow Trap

KV32 implements additional custom machine CSRs for stack safety accounting:

- `sguard_base` at `0x7CC` (lower stack bound; `0` disables guard checks)
- `spmin` at `0x7CD` (stack low-water mark)

KV32 also defines custom exception cause `16` to report stack overflow when SP is written below `sguard_base` while guard checks are enabled.

These CSRs and trap cause are KV32-specific and are not part of standard RISC-V privileged

architecture or `misa` extension bits.

D.5.3. Cacheability Fallback Rule

When no PMA region matches, KV32 applies a default architectural policy:

- `addr[31] = 1` → cacheable
- `addr[31] = 0` → non-cacheable

This fallback rule is not a RISC-V architectural requirement. It is a KV32 memory-system rule.

D.5.4. Machine-Mode-Only System Profile

KV32 is intentionally machine-mode only:

- No supervisor mode (`S`)
- No user mode (`U`)
- No virtual memory / page tables (`satp` absent)
- No delegation CSRs (`medeleg`, `mideleg` absent)
- No PMP CSRs; PMA is provided instead through the custom PMA block above

This makes KV32 a bare-metal / RTOS-oriented SoC profile rather than a Linux-class application processor.

D.6. Features That Are Platform-Specific, Not ISA Extensions

The following blocks are architecturally visible to software but should **not** be described as RISC-V ISA extensions:

- CLINT (`mtime`, `mtimecmp`, `msip`) — standard platform interrupt/timer block
- PLIC — standard platform interrupt controller model
- DMA controller
- GPIO controller
- Timer/PWM controller
- UART / SPI / I2C peripherals
- Watchdog timer
- Magic device for simulation console / exit control

These are SoC integration choices, not core ISA extensions.

D.7. Portability Guidance

For software targeting generic RISC-V systems:

1. Safe assumptions: `RV32IMAC + machine-mode traps + standard machine CSRs + atomic + compressed`.
2. Conditional assumptions: `FENCE.I`, cache maintenance operations, and cache hierarchy behavior.
3. KV32-only assumptions: PMA custom CSRs, PMA fallback cacheability rule, Magic device, and any software contract that depends on those features.

For software intended to be portable across RISC-V cores, isolate the following behind platform abstraction layers:

- PMA programming
- Stack-guard CSR programming and custom stack-overflow trap handling
- Cache maintenance helpers
- Simulation-only Magic device access
- Any firmware policy that assumes KV32's cacheability-by-address defaults

Appendix E: Revision History

Rev	Date	Author	Description
1.0	2026-02-25	KV32 Dev Team	Initial release
1.1	2026-02-28	KV32 Dev Team	Added GPIO (`axi_gpio.sv`) and Timer/PWM (`axi_timer.sv`) peripheral specifications; updated memory map and address decode (Slave 7=GPIO, Slave 8=Timer, Slave 9=Magic); corrected DMA channel count (8→4); added PLIC sources 5 (GPIO) and 6 (Timer); added CAP registers to UART/SPI/I2C/DMA/GPIO/Timer; expanded Appendix A/C accordingly
1.2	2026-03-09	KV32 Dev Team	Added Data Cache specification (Section 6): write-back/write-through, write-allocate/no-allocate, dirty eviction, CMO ops (INVAL/FLUSH/CLEAN/FLUSH_ALL); added Physical Memory Attributes (PMA) CSRs (`pmacfg0/1`, `pmaaddr0`–`pmaaddr7`, 0x7C0–0x7CB); updated SoC block diagram (SVG + ASCII) to show D-Cache; updated System Components, Key Features, Abstract, and Performance Optimization sections; added D-Cache burst transaction details to AXI section; updated power estimate to ~50 mW

Rev	Date	Author	Description
1.3	2026-03-14	KV32 Dev Team	Added Appendix D summarizing RISC-V architectural extensions and KV32-specific architecture-visible features; clarified distinction between standard ISA extensions, standard platform blocks, and KV32 custom PMA architecture; clarified I2C IS semantics: IS[2] STOP done is sticky W1C and IS[1:0] are level status